

Biometrische Personalisierung eines KI-Assistenten

Entwicklung eines kamerabasierten Systems zur Gesichtserkennung und Nutzeridentifikation

2. Projektarbeit

des Studienganges Informatik
an der DHBW Ravensburg

von Hannes Klas

Kurs: TIK24
Matrikelnummer: 5872179
Dualer Partner: SAP SE, 88677 Markdorf
Projektbetreuer: [Projektbetreuer]
Gutachter DHBW: [Gutachter DHBW]
Abgabedatum: [Abgabedatum]

Inhaltsverzeichnis

Abkürzungsverzeichnis.....	IV
Abbildungsverzeichnis.....	V
Tabellenverzeichnis.....	VI
1 Einleitung.....	1
1.1 Thematischer Hintergrund und Problemstellung.....	1
1.2 Einordnung in den Forschungsstand.....	1
1.3 Zielsetzung und Forschungsfrage.....	2
1.4 Aufbau der Arbeit.....	2
2 Grundlagen.....	3
2.1 Kamerabasierte Gesichtsdetektion.....	3
2.1.1 Neuronale Netze zur Gesichtsdetektion.....	3
2.1.2 Interaktionsvalidierung durch Vision-LLM.....	3
2.2 Biometrische Identifikation mit Gesichts-Embeddings.....	4
2.2.1 Embedding-Räume und Kosinus-Ähnlichkeit.....	4
2.2.2 Metrisches Lernen: Angular Margin Loss.....	5
2.3 Sitzungsübergreifendes Gedächtnis in LLM-Systemen.....	5
2.3.1 Das Kontextfenster-Problem.....	5
2.3.2 Gesprächszusammenfassung als Langzeitgedächtnis.....	5
2.3.3 Retrieval-Augmented Generation.....	6
3 Konzeption.....	7
3.1 Gesamtarchitektur und Systemüberblick.....	7
3.1.1 Anforderungsanalyse.....	8
3.2 Auswahl des Detektionsansatzes.....	8
3.3 Auswahl des Erkennungsmodells.....	9
3.4 Auswahl der Persistenzschicht.....	11
3.5 Datenschutz und biometrische Daten.....	12
3.6 Wirtschaftliche Bewertung.....	12
3.7 Nachhaltigkeitsaspekte.....	13
4 Personenerkennung.....	14
4.1 Gesichtsdetektion mit MediaPipe BlazeFace.....	14
4.2 Gaze-Validierung via Vision-LLM.....	14
4.3 State Machine: IDLE → CANDIDATE → ACTIVE.....	15
4.4 Paralleles Multi-Person-Tracking und Gruppen-Erkennung.....	16
5 Biometrische Identifikation und Persistenz.....	17
5.1 Biometrische Identifikation mit InsightFace.....	17
5.2 Zweistufiges Tracking und Embedding-Stabilisierung.....	18

5.3 Qdrant-Persistenz und EMA-Blending.....	19
6 Sitzungsübergreifende Personalisierung.....	20
6.1 Strukturierte Fakt-Extraktion.....	20
6.2 RAG-Retrieval und Kontext-Injektion.....	21
6.3 Gruppen-Sessions und History-Isolation.....	23
7 Evaluation.....	24
7.1 Erkennungsgenauigkeit.....	24
7.2 Systemlatenz.....	25
7.3 Robustheit.....	26
7.4 Personalisierungsqualität.....	26
8 Fazit und Ausblick.....	27
8.1 Fazit.....	27
8.2 Ausblick.....	28
Anhang.....	29
Literaturverzeichnis.....	31

Abkürzungsverzeichnis

ANN	Approximate Nearest Neighbor
API	Application Programming Interface
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DSGVO	Datenschutz-Grundverordnung
EMA	Exponential Moving Average
GPU	Graphics Processing Unit
HNSW	Hierarchical Navigable Small World
HTTP	Hypertext Transfer Protocol
K8s	Kubernetes
LFW	Labeled Faces in the Wild
LLM	Large Language Model
MCP	Model Context Protocol
ONNX	Open Neural Network Exchange
RAG	Retrieval-Augmented Generation
SAP	Systeme, Anwendungen und Produkte
SIMD	Single Instruction, Multiple Data
STT	Speech-to-Text
TTS	Text-to-Speech
UX	User Experience
VAD	Voice Activity Detection
VLM	Vision-Language-Modell

Abbildungsverzeichnis

Abbildung 1	Systemarchitektur: vier Microservices mit definierten Kommunikationsschnittstellen.....	7
Abbildung 2	Zustandsdiagramm der PresenceStateMachine.....	15
Abbildung 3	Zweistufiger Tracking-Algorithmus: Positions-Präfilter vor ArcFace-Matching.	18
Abbildung 4	RAG-Retrieval-Flow: Embedding der Anfrage, Qdrant-Suche, Kontext-Injektion.	21

Tabellenverzeichnis

Tabelle 1	Anforderungsanalyse: Messbare Systemanforderungen vor den Designentscheidungen.....	8
Tabelle 2	Vergleich evaluierter Gesichtsdetektionsansätze.....	9
Tabelle 3	Vergleich evaluierter biometrischer Erkennungsmodelle (* reale Genauigkeit im DeepFace-Framework; eigene Beobachtung).....	10
Tabelle 4	Vergleich evaluierter Persistenzlösungen für biometrische Embeddings.	11
Tabelle 5	Geschätzte API-Kosten im 8-Stunden-Kiosk-Betrieb (30 Besucher/Tag). * Die Sprachmodell-Kosten fallen in allen Szenarien gleich an und werden daher nicht verglichen; verglichen wird nur die biometrische Identifikation. AWS Rekognition: \$0,001/Bild; Azure Face API: \$1,50/1.000 Transaktionen (laut öffentlichem Listenpreis).....	13
Tabelle 6	Dreikanaliges Gedächtnisdesign: Persistenzfelder der Fakt-Extraktion.	20
Tabelle 7	Soll-Ist-Abgleich der Anforderungen A-01 bis A-04 aus Kap. 3.1.....	24
Tabelle 8	Systemlatenz: sequenzielle Wartezeit und parallele Verarbeitung.....	25
Tabelle 9	Robustheitsfaktoren und Gegenmaßnahmen im Testbetrieb.....	26
Tabelle 10	Übersicht über die Nutzung von KI-basierten Werkzeugen.....	29
Tabelle 11	Konfigurierbare Parameter des Presence Service mit Standardwert, Einheit und Primärkapitel.....	29
Tabelle 12	Wirkung der BlazeFace-Detektionsfilter und Gaze-Validator-Konfiguration.....	30
Tabelle 13	Kennwerte des InsightFace buffalo_I Erkennungsmodells und der FaceStore-Persistenzschicht.....	30

1 Einleitung

1.1 Thematischer Hintergrund und Problemstellung

Öffentliche KI-Assistenten — etwa sprachgesteuerte Kiosk-Systeme im Empfang von Unternehmen oder auf Messen — kommen bewusst ohne Login aus (Porcheron *u. a.*, 2018, S. 3). Das ist eine Designentscheidung: Jeder Besucher soll das System sofort nutzen können. Es gibt keine Tastatur und keine Login-Maske, also kann sich der Nutzer auch nicht selbst zu erkennen geben (Jain, Ross und Nandakumar, 2011, S. 1). Diese Offenheit hat aber einen Nachteil: Das System weiß nicht, mit wem es spricht, und kann seine Antworten kaum auf die einzelne Person zuschneiden.

Der Grund liegt in der Funktionsweise großer Sprachmodelle (LLMs): Sie haben kein Gedächtnis, das über eine einzelne Sitzung hinausreicht (Zhang *u. a.*, 2018, S. 2388). Was ein Nutzer heute sagt, ist für das Modell beim nächsten Besuch wieder unbekannt — es kennt weder seinen Namen noch seine Interessen noch frühere Gespräche (Zhang *u. a.*, 2018, S. 2390). Für echte Personalisierung müsste der gespeicherte Kontext eines Besuchers zu Sitzungsbeginn wieder eingespielt werden, und das setzt voraus, dass das System die Person wiedererkennt.

Dazu kommt eine praktische Grenze: Man kann nicht einfach die komplette Gesprächshistorie in jeden Aufruf laden. Das Kontextfenster eines LLM — die Textmenge, die es gleichzeitig verarbeiten kann — ist begrenzt, und je länger die Historie wird, desto unzuverlässiger nutzt das Modell die relevanten Stellen mitten im Text (Liu *u. a.*, 2024, S. 1). Ein sinnvolles Verfahren muss deshalb gezielt nur die Inhalte abrufen, die zur aktuellen Frage passen.

Die Aufgabe dieser Arbeit besteht damit aus zwei verbundenen Teilen: Zuerst muss die Person erkannt werden, danach lässt sich passender Kontext aus früheren Sitzungen abrufen — beides ohne aktives Zutun des Nutzers. Für die Erkennung bietet sich kamerabasierte Gesichtserkennung an, das gängigste Verfahren, um Personen berührungslos zu identifizieren (Guo und Zhang, 2019, S. 4). Erst die erkannte Identität macht es möglich, gespeicherten Kontext der richtigen Person zuzuordnen (Jain, Ross und Nandakumar, 2011, S. 12). Wie sich beide Bausteine in einem öffentlichen KI-Assistenten zusammenführen lassen, ist der Gegenstand dieser Arbeit.

1.2 Einordnung in den Forschungsstand

Bestehende Ansätze zur Personalisierung von Sprachassistenten setzen meist auf feste Nutzerprofile mit Login (Zhang *u. a.*, 2018, S. 2388–2390) oder speichern Kontext nur kurzzeitig innerhalb einer Sitzung (Xu, Szlam und Weston, 2022, S. 1). Für einen öffentlichen Kiosk ohne Anmeldung fällt die erste Variante weg (Porcheron *u. a.*, 2018, S. 3), und die zweite reicht nicht

über den einzelnen Besuch hinaus. Biometrische Identifikation ohne vorherige Registrierung ist zwar aus Zugangskontrollen bekannt (Jain, Ross und Nandakumar, 2011, S. 12), wurde aber bisher nicht mit einer sitzungsübergreifenden Gesprächspersonalisierung verbunden. Auch Systeme, die Kontext gezielt auslagern und über Sitzungen hinweg vorhalten (etwa MemGPT), setzen eine bereits bekannte Nutzeridentität voraus (Packer u. a., 2024, S. 1–2). Genau hier setzt diese Arbeit an: Die Identität kommt passiv über das Gesicht, ohne aktive Anmeldung, gekoppelt mit einem Gesprächsgedächtnis, das relevante Inhalte gezielt nachlädt (Retrieval-Augmented Generation, kurz RAG) (Lewis u. a., 2020, S. 2) — ohne Login, ohne GPU-Server und im Rahmen der DSGVO.

1.3 Zielsetzung und Forschungsfrage

Die Arbeit untersucht, wie ein öffentlicher KI-Assistent wiederkehrende Nutzer per Gesichtserkennung erkennen und ihnen über mehrere Sitzungen hinweg personalisierte Antworten geben kann. Im Mittelpunkt stehen Konzeption, Umsetzung und Bewertung eines vollständigen Prototyps — von der Erkennung über die Speicherung bis zur personalisierten Sitzung. Die Forschungsfrage lautet:

„Wie kann ein öffentlicher KI-Assistent durch kamerabasierte Gesichtserkennung mittels Deep-Learning-Embeddings wiederkehrende Nutzer identifizieren und eine sitzungsübergreifend personalisierte Konversationserfahrung bereitstellen?“

Die technische Grundlage — Gesichtserkennung über Deep-Learning-Embeddings (Deng u. a., 2019, S. 1) (numerische Merkmalsvektoren, die ein Gesicht vergleichbar machen) und RAG zur Kontextbereitstellung (Lewis u. a., 2020, S. 2) — wird in Kap. 2 theoretisch eingeordnet und in Kap. 3 als Architekturentscheidung begründet.

1.4 Aufbau der Arbeit

Die Arbeit gliedert sich in acht Kapitel. Kap. 2 legt die theoretischen Grundlagen: Gesichtsdetektion, biometrische Identifikation über Gesichts-Embeddings und sitzungsübergreifendes Gedächtnis in LLM-Systemen. Kap. 3 stellt die Gesamtarchitektur vor und begründet die zentralen Technologieentscheidungen samt Datenschutz- und Wirtschaftlichkeitsaspekten. Kap. 4 beschreibt die Personenerkennung, Kap. 5 die biometrische Identifikation und Profilspeicherung, Kap. 6 die sitzungsübergreifende Personalisierung samt Datenschutz bei Gruppen-Sessions. Kap. 7 bewertet das System nach Genauigkeit, Latenz und Robustheit und benennt die Grenzen der Evaluation; Kap. 8 beantwortet die Forschungsfrage und gibt einen Ausblick.

2 Grundlagen

Dieses Kapitel legt den theoretischen Rahmen für die in den Kapiteln 4 bis 6 beschriebene Implementierung. Im Vordergrund stehen drei Themenbereiche, die unmittelbar in den entwickelten Komponenten Anwendung finden: kamerabasierte Gesichtsdetektion (Kap. 2.1), biometrische Identifikation über Gesichts-Embeddings (Kap. 2.2) sowie sitzungsübergreifendes Gedächtnis in LLM-basierten Systemen (Kap. 2.3). Eingeführt werden nur Konzepte, die für spätere Implementierungsentscheidungen relevant sind.

2.1 Kamerabasierte Gesichtsdetektion

Die automatische Erkennung von Gesichtern in Echtzeit bildet die Grundlage für die Anwesenheitserkennung. Dieser Abschnitt beschreibt die neuronalen Gesichtsdetektoren sowie die Methode, mit der das System entscheidet, ob eine erkannte Person aktiv mit dem Kiosk interagieren möchte.

2.1.1 Neuronale Netze zur Gesichtsdetektion

Das System nutzt BlazeFace als Gesichtsdetektor. Dabei handelt es sich um ein neuronales Netz, das für mobile Geräte optimiert ist und ein Bild in einem einzigen Durchlauf nach Gesichtern absucht (Bazarevsky *u. a.*, 2019, S. 2–3). Es ist gezielt auf frontale Gesichter ausgelegt und arbeitet dadurch sehr schnell. Für den Kiosk mit fester Kameraposition und konstantem Abstand zur Person ist diese Ausrichtung auf Frontalgesichter kein Nachteil, sondern passt genau zum Einsatzzweck. BlazeFace ist Teil des MediaPipe-Frameworks, einer Sammlung fertiger Bausteine für die Bildverarbeitung in Echtzeit (Lugaresi *u. a.*, 2019, S. 1–2).

2.1.2 Interaktionsvalidierung durch Vision-LLM

Das System verzichtet auf klassische Blickrichtungsschätzung (Gaze Estimation), weil sie für jeden neuen Nutzer eine personenspezifische Kalibrierung erfordert; ohne sie verschlechtern sich die Ergebnisse unter realen Bedingungen deutlich (Cheng *u. a.*, 2024, §1, S. 1–2), (Zhang *u. a.*, 2015, S. 1–2). In einem öffentlichen Kiosk mit ständig wechselnden Besuchern lässt sich dieser Schritt nicht durchführen.

Stattdessen fällt die Entscheidung, ob eine Person mit dem Kiosk interagieren möchte, über ein einfaches Ja/Nein-Urteil eines Vision-Sprachmodells: schaut die Person in die Kamera oder nicht. Solche Modelle (etwa Gemini 2.5 Flash) verarbeiten Bild und Sprache gemeinsam und lösen eine Aufgabe allein anhand einer Beschreibung in normaler Sprache — ohne vorheriges Training für genau diese Aufgabe und ohne Beispielbilder (Yin *u. a.*, 2024, S. 6729–6730).

Das passt zum Kiosk mit wechselnden, unbekannten Personen: keine Kalibrierung pro Nutzer, keine Trainingsbeispiele.

2.2 Biometrische Identifikation mit Gesichts-Embeddings

Die Wiedererkennung einer Person über mehrere Sitzungen hinweg braucht ein Merkmal, das eindeutig zur Person passt, sich kompakt speichern und schnell vergleichen lässt. Gesichts-Embeddings — kompakte Zahlenvektoren, die ein Gesichtsbild beschreiben — erfüllen das und bilden die Grundlage des eingesetzten Identifikationsverfahrens.

2.2.1 Embedding-Räume und Kosinus-Ähnlichkeit

Das Identifikationsverfahren beruht auf dem Konzept des Embedding-Raums: Ein Convolutional Neural Network (CNN — ein auf Bildverarbeitung spezialisiertes neuronales Netz) übersetzt jedes Gesichtsbild in einen Vektor, und der Abstand zwischen zwei solchen Vektoren gibt an, wie ähnlich sich die Gesichter sind (Taigman *u. a.*, 2014, S. 1–4). Entscheidend für die Wiedererkennung ist dabei eine einfache Eigenschaft: Gesichter derselben Person liegen im Vektorraum nah beieinander, Gesichter verschiedener Personen weit auseinander. Genau darauf setzt FaceNet (Schroff, Kalenichenko und Philbin, 2015, S. 1). FaceNet misst den Abstand als euklidische Distanz; das hier eingesetzte Verfahren nutzt stattdessen die Kosinus-Ähnlichkeit, also den Winkel zwischen den Vektoren.

Als Ähnlichkeitsmaß zwischen zwei Embeddings a und b dient die Kosinus-Ähnlichkeit. Sie misst den Winkel zwischen den Vektoren und ist damit unabhängig von deren Länge:

$$\cos(\theta) = \frac{a \cdot b}{\|a\| \cdot \|b\|} \quad (1)$$

Sind die Embeddings auf Länge 1 normiert (L2-Normierung), vereinfacht sich das zum Skalarprodukt:

$$\cos(\theta) = a \cdot b \quad (2)$$

Über einen Schwellwert auf diesem Kosinus-Wert lässt sich dann entscheiden, ob zwei Gesichtsbilder dieselbe Person zeigen. Wo der Schwellwert liegt, bestimmt das Verhältnis zwischen fälschlich akzeptierten und fälschlich abgewiesenen Personen und muss für die jeweilige Anwendung kalibriert werden.

Bei vielen gespeicherten Gesichtern wäre ein Vergleich mit jedem einzelnen Vektor zu langsam. Das eingesetzte Qdrant-Backend nutzt deshalb HNSW, ein Verfahren, das sehr schnell die ähnlichsten Vektoren findet, ohne alle durchzurechnen (Malkov und Yashunin, 2020, S. 1–2). Die Details dazu folgen in Kap. 5.

2.2.2 Metrisches Lernen: Angular Margin Loss

Ein einfach trainiertes Netz lernt vor allem, die Trainingsbilder korrekt einzusortieren. Es stellt aber nicht sicher, dass Gesichter derselben Person im Vektorraum eng zusammenliegen — und genau darauf kommt es bei der Wiedererkennung an. Margin-basierte Verlustfunktionen setzen hier an: Sie zwingen das Netz beim Training, zwischen den Personen einen festen Sicherheitsabstand einzuhalten (Liu *u. a.*, 2017, S. 1–3), (Wang *u. a.*, 2018, S. 1–2). Das hier verwendete ArcFace erzwingt diesen Abstand als festen Winkelabstand (Deng *u. a.*, 2019, S. 3); das Ergebnis sind kompaktere Gruppen derselben Person und größere Abstände zwischen verschiedenen Personen — genau die Eigenschaft, die der spätere Schwellwertvergleich (Kap. 2.2.1) braucht. CosFace erreicht denselben Effekt, indem der Abstand auf den Kosinus-Wert statt auf den Winkel angesetzt wird (Wang *u. a.*, 2018, S. 1–2).

2.3 Sitzungsübergreifendes Gedächtnis in LLM-Systemen

Konversationelle KI-Systeme, die über mehrere Sitzungen hinweg personalisiert antworten sollen, stehen vor einer grundlegenden Hürde: Sprachmodelle haben kein dauerhaftes Gedächtnis. Jede Sitzung beginnt mit einem leeren Kontextfenster. Dieser Abschnitt erläutert das Kontextfenster-Problem, zusammenfassungsbasierte Gedächtnisansätze und Retrieval-Augmented Generation.

2.3.1 Das Kontextfenster-Problem

Das Kontextfenster eines Sprachmodells ist sein Arbeitsgedächtnis: Nur was im Prompt steht, kann das Modell bei der Antwort nutzen. Selbst wenn der Platz ausreicht, nutzt das Modell den Inhalt nicht gleichmäßig — Information aus der Mitte langer Kontexte wird deutlich schlechter abgerufen als am Anfang oder Ende. Dieser Effekt ist als „Lost in the Middle,“ bekannt (Liu *u. a.*, 2024, S. 4–6). Einfach alle vergangenen Sitzungen in den Prompt zu packen, ist damit keine tragfähige Lösung für sitzungsübergreifende Personalisierung.

2.3.2 Gesprächszusammenfassung als Langzeitgedächtnis

Statt die gesamte rohe Gesprächshistorie mitzuschleppen, ist es sinnvoller, vergangene Sitzungen zusammenzufassen und zu verdichten. Systeme, die zusammenfassen und gezielt abrufen, kommen bei langen Gesprächsverläufen deutlich besser zurecht als Modelle, die nur die reine Historie verarbeiten (Xu, Szlam und Weston, 2022, S. 1). Alternativ lassen sich auch einzelne Fakten gezielt herausziehen — etwa Vorlieben, Gewohnheiten oder wiederkehrende Themen — was die gespeicherten Informationen später leichter auffindbar macht.

Die Grundidee, Informationen aus dem begrenzten Kontextfenster in einen externen Speicher auszulagern, greift das System von MemGPT auf (Packer *u. a.*, 2024, S. 1–3), setzt sie aber

über die gezielte Extraktion von Fakten und deren späteren Abruf per RAG um (Kap. 6.1 und 6.2).

2.3.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) verbindet ein Sprachmodell, dessen Wissen fest in den Modellgewichten steckt, mit einem externen, jederzeit aktualisierbaren Wissensspeicher. Der Ablauf ist zweistufig: Zuerst holt ein Retriever passende Dokumente aus dem Vektorindex, dann stützt der Generator seine Antwort auf diese Inhalte. So werden die Antworten konkreter und faktentreuer (Lewis *u. a.*, 2020, S. 4), und das externe Wissen lässt sich austauschen, ohne das Modell neu zu trainieren (Guu *u. a.*, 2020, S. 1–3) — für nutzerspezifische Informationen besonders wichtig.

Die Abrufseite arbeitet mit einer semantischen Ähnlichkeitssuche im Embedding-Raum — nach demselben Prinzip wie die Kosinus-Ähnlichkeit aus Kap. 2.2.1: Anfrage und Dokument werden in denselben Vektorraum übersetzt, thematische Nähe zeigt sich im Abstand der Vektoren. Das System nutzt dafür all-MiniLM-L12-v2 (SBERT-Architektur, 384-dimensionale Vektoren). Der konkrete Einsatz folgt in Kap. 6.2 und 6.3.

3 Konzeption

Dieses Kapitel begründet die zentralen Designentscheidungen des Systems. Es geht dabei nicht darum, das Implementierte zu beschreiben, sondern nachvollziehbar herzuleiten, warum diese Technologien gewählt wurden. Kap. 3.1 gibt den Architekturüberblick, Kap. 3.2 bis 3.4 begründen die drei Technologieentscheidungen (Detektion, Erkennungsmodell, Persistenz), Kap. 3.5 bis 3.7 behandeln Datenschutz, Wirtschaftlichkeit und Nachhaltigkeit. Die Implementierungsdetails folgen in den Kapiteln 4 bis 6.

3.1 Gesamtarchitektur und Systemüberblick

Das System besteht aus vier lose gekoppelten Diensten nach dem Microservice-Prinzip: Jeder Dienst ist unabhängig deploybar, hat klar abgegrenzte Aufgaben und kommuniziert über feste Schnittstellen (Dragoni *u. a.*, 2017, S. 1–3). So lässt sich eine Komponente — etwa das Erkennungsmodell oder die Persistenzschicht — austauschen, ohne den Rest anzufassen. Die Wahrnehmungspipeline des Presence Service baut auf dem quelloffenen MediaPipe-Framework auf, das stellvertretend für den Open-Source-Stack steht (Lugaresi *u. a.*, 2019, S. 1–2). Die vier Dienste sind das Frontend (React, Port 5173), das Backend (FastAPI, Port 8000), der MCP Panel Server (FastMCP, Port 8001) und der Presence Service (Python/MediaPipe, Port 8002).

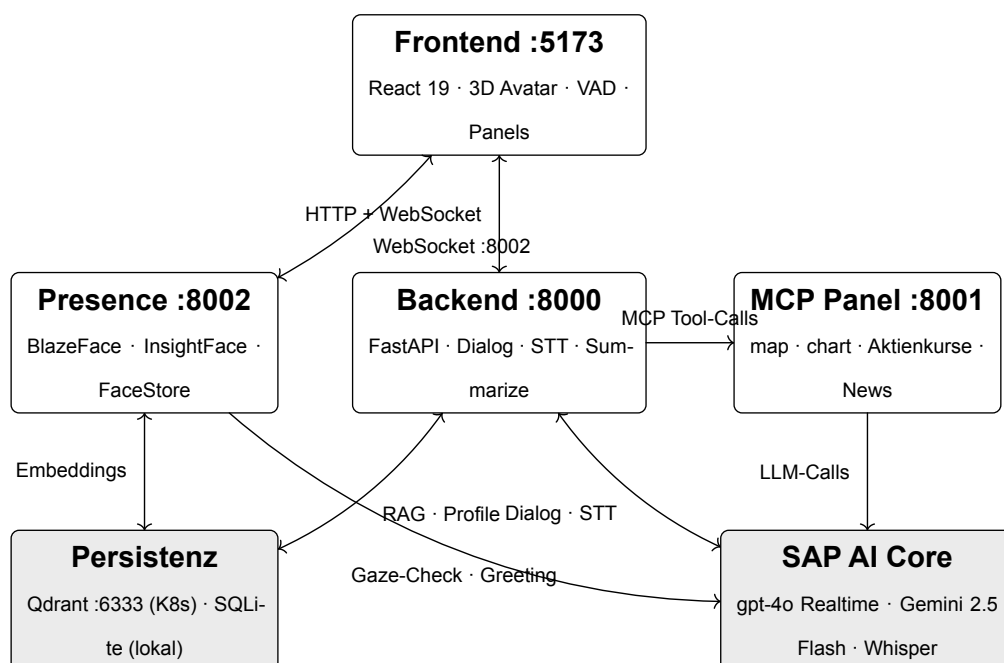


Abbildung 1: Systemarchitektur: vier Microservices mit definierten Kommunikationsschnittstellen

Die drei Hauptdatenflüsse verlaufen über HTTP und WebSocket zwischen Frontend und Backend, über MCP-Tool-Calls zwischen Backend und MCP Panel Server sowie über eine WebSocket-Verbindung zwischen Presence Service und Frontend für Presence-Events.

Der Presence Service ist die Kernkomponente: Er verarbeitet den Kamerastream, führt Gesichtsdetektion, Tracking und Identifikation durch und speichert die Nutzerprofile. SAP AI Core stellt die Sprachmodelle bereit: Das Gespräch — Spracheingabe und -ausgabe — läuft über ein Realtime-Sprachmodell (gpt-4o Realtime), Gemini 2.5 Flash übernimmt die Bild- und Textaufgaben (Gaze-Check, Begrüßung, Sitzungszusammenfassung). Ein Whisper-Modell erzeugt zusätzlich ein Transkript der Nutzeräußerung, das den Kontextabruf für das Gesprächsgedächtnis auslöst (vgl. Kap. 6.2).

Jede erkannte Person wird über eine `PresenceStateMachine` verwaltet, die drei Zustände kennt: *IDLE* (Person nicht aktiv erkannt), *CANDIDATE* (Person erkannt, Interaktionsabsicht wird per Gaze-Check geprüft) und *ACTIVE* (Gaze-Check positiv, Sitzung läuft). Der interne Verarbeitungsfluss — von der Kamera über Detektion, Tracking und Identifikation bis zum WebSocket-Broadcast an das Frontend — wird in den Kapiteln 4 und 5 im Detail beschrieben; Kap. 4.3 zeigt den Zustandsautomaten vollständig.

3.1.1 Anforderungsanalyse

Die folgende Tabelle fasst die messbaren Systemanforderungen zusammen, gegen die die Designentscheidungen der nächsten Abschnitte geprüft werden:

ID	Anforderung	Zielwert	Erfüllt durch
A-01	Embedding-Latenz auf Standard-CPU	≤ 100 ms	InsightFace buffalo_I via ONNX (80 ms, vgl. Kap. 3.3)
A-02	Erkennungsgenauigkeit (LFW-Benchmark)	≥ 99 %	ArcFace ResNet50: 99,83 % (vgl. Kap. 3.3)
A-03	Infrastruktur CPU-only, kein GPU-Server	Hard Constraint	ONNX Runtime CPUExecution-Provider (vgl. Kap. 3.3)
A-04	Keine Falschakzeptanz im Betrieb	keine im Test	Schwellenwert-Kalibrierung 0,52 (vgl. Kap. 5.1, Kap. 7.1)

Tabelle 1: Anforderungsanalyse: Messbare Systemanforderungen vor den Designentscheidungen

Die Anforderungen A-01 bis A-03 folgen aus den Einsatzbedingungen am Kiosk; A-04 ergänzt die Sicherheitsanforderung, die für biometrische Identifikation im öffentlichen Raum wichtig ist. Ob jede Anforderung erfüllt wird, weisen die folgenden Entscheidungsabschnitte nach (vgl. Kap. 3.2–3.4).

3.2 Auswahl des Detektionsansatzes

Am Kiosk soll das System nur auf Personen reagieren, die aktiv davor stehen. Wer vorbeigeht oder seitlich zur Kamera steht, darf es nicht auslösen. Die Kamera ist fest verbaut, und ein echter Nutzer nimmt bewusst Blickkontakt zum Gerät auf. Eine Seiten- oder Profilsicht deutet

deshalb meist auf einen Passanten hin, nicht auf einen Nutzer. Entscheidend für die Modellwahl ist damit, dass der Detektor bevorzugt Frontalgesichter erkennt.

Die folgende Tabelle vergleicht die evaluierten Detektionsansätze anhand dieser Kriterien:

Ansatz	Erkannte Winkel	CPU-Latenz	Fehlauslösungen	Ergebnis
MediaPipe Blaze-Face	Nur frontal	15–30 ms	Keine Seitenansichten	Gewählt
YOLO	Alle Winkel, auch Seitenansichten	20–30 ms	Hintergrundpersonen, Vorbeigehende	Verworfen
Vision-LLM (Gemini)	Nicht für kontinuierliche Detektion geeignet	1–2 s	Zu hohe Latenz für Frame-Scanning	Nicht evaluiert

Tabelle 2: Vergleich evaluierter Gesichtsdetektionsansätze

YOLO-basierte Detektoren erkennen Gesichter aus allen Winkeln, auch Seitenansichten, und sind damit ungeeignet: In eigenen Tests löste YOLO regelmäßig bei Personen aus, die nur vorbeigingen oder seitlich im Bild standen. MediaPipe BlazeFace ist dagegen als frontaler Detektor ausgelegt, sodass seitlich stehende oder vorbeigehende Personen von vornherein nicht erkannt werden (Bazarevsky *u. a.*, 2019, S. 2–3). Mit ca. 15–30 ms CPU-Latenz pro Frame reicht das für Echtzeit-Verarbeitung auf Standard-Hardware ohne GPU (Lugaresi *u. a.*, 2019, S. 1–2).

Als zweite Filterschicht prüft ein Vision-LLM (Gemini 2.5 Flash) den Blickkontakt. Die naheliegende Alternative wäre klassische Gaze-Estimation (Schätzung der Blickrichtung), die aber eine Kalibrierungssitzung pro Nutzer voraussetzt (Kellnhofer *u. a.*, 2019, S. 1–2), (Cheng *u. a.*, 2024, §2, S. 2–4). An einem öffentlichen Kiosk mit wechselnden, unbekannten Passanten ist das nicht durchführbar — ohne Kalibrierung verliert das Verfahren unter realen Bedingungen deutlich an Genauigkeit (Cheng *u. a.*, 2024, §1, S. 1–2). Gemini klassifiziert das Kamerabild dagegen direkt, ohne nutzerspezifische Kalibrierung (Yin *u. a.*, 2024, S. 1–3), (Radford *u. a.*, 2021, S. 1–3).

3.3 Auswahl des Erkennungsmodells

Das biometrische Erkennungsmodell bestimmt Genauigkeit und Latenz des Identifikationssystems. Die Anforderungen für den Kiosk sind klar: hohe Erkennungsgenauigkeit bei niedriger False-Accept-Rate (Fremde dürfen nicht als bekannte Nutzer durchgehen), CPU-Inferenz ohne GPU-Server, keine Cloud-Abhängigkeit für biometrische Daten und ONNX-Kompatibilität für

schlankes Deployment. Diese Kombination schließt viele cloudbasierte oder GPU-abhängige Modelle von vornherein aus.

Die folgende Tabelle stellt die evaluierten Modelle anhand dieser Kriterien gegenüber:

Modell	LFW-Gen.	CPU-Latenz	Stack	Ergebnis
InsightFace buffalo_l (ArcFace ResNet50)	99,83 %	80 ms	ONNX Runtime	Gewählt
InsightFace buffalo_s (ArcFace MobileNet)	99,5 %	20 ms	ONNX Runtime	Nicht gewählt
DeepFace + ArcFace	92 % real*	300 ms	TensorFlow/Keras	Verworfen
DeepFace + FaceNet512	97 % real*	250 ms	TensorFlow/Keras	Verworfen
OpenCV SFace	99,3 %	10 ms	ONNX Runtime	Nicht gewählt
dlib face_recognition	99,38 %	150 ms	C++/dlib	Nicht gewählt

Tabelle 3: Vergleich evaluierter biometrischer Erkennungsmodelle (* reale Genauigkeit im DeepFace-Framework; eigene Beobachtung)

Alle betrachteten Modelle setzen das Embedding-Raumkonzept um (Kap. 2.2.1) — sie unterscheiden sich nicht im *Was*, sondern im *Wie*: in der Verlustfunktion, mit der sie trainiert wurden, und im Inferenz-Stack, auf dem sie laufen (Schroff, Kalenichenko und Philbin, 2015, S. 1–2), (Taigman *u. a.*, 2014, S. 1701–1703).

Ausgangspunkt der Evaluation war das DeepFace-Framework, das eine schnelle erste Integration erlaubte. Im Entwicklungsbetrieb erreichte es unter realen Bedingungen aber nur ca. 92 % Genauigkeit bei ca. 300 ms Latenz pro Embedding und erfüllte die Echtzeit-Anforderung nicht. Das gab den Ausschlag für den Wechsel zu InsightFace buffalo_l (ArcFace ResNet50): 99,83 % LFW-Genauigkeit bei ca. 80 ms CPU-Latenz (eigene Messung). Der Genauigkeitsvorsprung geht auf die ArcFace-Verlustfunktion zurück (vgl. Kap. 2.2.2) (Deng *u. a.*, 2019, S. 3–5), (Guo und Zhang, 2019, S. 3–5). Die niedrigere Latenz ist relevant, weil der Presence Service im Takt von 1,0 s scannt (vgl. Kap. 4.1) und das Embedding innerhalb dieses Intervalls fertig sein muss.

Die übrigen Alternativen scheiden entlang derselben zwei Dimensionen aus: buffalo_s wurde trotz geringerer Latenz verworfen, da schon kleine Genauigkeitseinbußen die False-Accept-Rate erhöhen; die weiteren Modelle liegen in Genauigkeit, Latenz oder beidem unter buffalo_l (vgl. Tabelle 3). Die Inferenz über ONNX Runtime läuft CPU-optimiert ohne GPU-Server und spart die Latenz von Cloud-Calls pro Frame.

3.4 Auswahl der Persistenzschicht

Die Persistenzschicht speichert die Gesichtsembeddings und das Gesprächsgedächtnis. Ein FaceStore-Interface entkoppelt diese Logik vom Tracking-Code, sodass sich das Backend per Umgebungsvariable wechseln lässt. Drei Lösungen wurden evaluiert:

Persistenzlösung	K8s-fähig	ANN-Index	Embedding-Dim.	Ergebnis
Qdrant	Ja	HNSW (CO-SINE)	512-dim (ArcFace) + 384-dim (RAG)	Gewählt
SAP HANA Cloud Vector Engine	Ja (Cloud-managed)	Proprietär	Flexibel	Nicht gewählt
SQLite	Nein (POSIX-Lock)	Kein ANN-Index	Beliebig	Nur lokal

Tabelle 4: Vergleich evaluierter Persistenzlösungen für biometrische Embeddings

Das FaceStore-ABC gibt allen Backends eine einheitliche Schnittstelle mit `find_profile`, `upsert_profile` und `find_relevant_chunks` vor. Damit hängt der Tracker-Code nur von der abstrakten Schnittstelle ab, nicht von einer konkreten Datenbank (Dependency-Inversion-Prinzip) (Martin, 2017, S. 65–70); der Wechsel zwischen lokaler und Produktiv-Persistenz erfolgt per Umgebungsvariable.

Für die lokale Entwicklung dient SQLite als dateibasiertes Backend ohne Setup. Im Kubernetes-Deployment auf Azure ist SQLite nicht einsetzbar und wird durch Qdrant ersetzt. Als K8s-native Alternative wurde SAP HANA Cloud Vector Engine geprüft, aber nicht gewählt: HANA ist primär relational, deren Vektorsuche als Erweiterung dazugekommen ist — für einen Fall, der ausschließlich Vektoren speichert und abfragt, ist das mehr Funktionsumfang als nötig. Qdrant ist dagegen von Grund auf dafür gebaut: schlankere Konfiguration, direkt zugänglicher HNSW-Index und in der Praxis erprobt. Die Entscheidung fiel damit nicht gegen SAP-Infrastruktur, sondern für das passende Werkzeug beim Prototyp.

Qdrant ist eine vektornative Datenbank und läuft als Kubernetes-Pod. Sie nutzt zwei Collections: `face_profiles` mit 512-dimensionalen ArcFace-Embeddings für die Identifikation und `conversation_chunks` mit 384-dimensionalen RAG-Embeddings (all-MiniLM-L12-v2) für das Gesprächsgedächtnis. Als Index dient HNSW (Kap. 2.2.1), der die im Live-Betrieb nötige logarithmische Suchkomplexität liefert (Malkov und Yashunin, 2020, S. 1–2), (Johnson, Douze und Jégou, 2019, S. 1–3). Die Suchzeit bleibt dadurch auch bei vielen tausend Profilen unkritisch, zumal die Embedding-Berechnung (80 ms, vgl. Kap. 3.3) die Gesamtlatenz dominiert. Die `conversation_chunks`-Collection dient als Gedächtnis im Sinne des in Kap. 2.3.3 hergelei-

teten RAG-Mechanismus (Lewis *u. a.*, 2020, S. 4–5), (Guu *u. a.*, 2020, S. 1–3); ihre Nutzung beschreibt Kap. 6.2.

3.5 Datenschutz und biometrische Daten

Gesichtsembeddings sind biometrische Daten im Sinne von Art. 9 Abs. 1 DSGVO, da sie aus einer natürlichen Person abgeleitet werden und zur eindeutigen Identifikation taugen (Europäisches Parlament und Rat der Europäischen Union, 2016, Art. 9 Abs. 1). Solche Daten besonderer Kategorie unterliegen einem strengen Rechtsrahmen (Voigt und Bussche, 2017, S. 114–120), (Krivokuca Hahn und Marcel, 2023, S. 639–641).

Das System ist ein interner SAP-Prototyp und wird nicht für Endkunden deployt. Die biometrischen Daten werden ausschließlich im internen Netzwerk verarbeitet: Der Qdrant-Pod läuft im SAP-eigenen Kubernetes-Cluster, und die Embeddings verlassen diesen Cluster nicht. Alle Testpersonen haben eingewilligt.

Für ein reales Kundensystem wären zusätzliche Maßnahmen nötig: ein explizites Opt-in nach Art. 9 Abs. 2 lit. a DSGVO, das Recht auf Löschung gespeicherter Embeddings nach Art. 17 DSGVO (Europäisches Parlament und Rat der Europäischen Union, 2016, Art. 17) und eine Datenschutz-Folgenabschätzung nach Art. 35 DSGVO (Europäisches Parlament und Rat der Europäischen Union, 2016, Art. 35; Voigt und Bussche, 2017, S. 152–160). Diese liegen außerhalb des Prototyp-Scopes. Hinzu kommt die gesellschaftliche Akzeptanz: Nutzer akzeptieren biometrische Identifikation eher, wenn Zweck und Datenspeicherung transparent sind (Rotter, 2008, S. 68–70). Ein öffentliches Deployment müsste einen transparenten Kamerahinweis und eine klare Opt-out-Möglichkeit früh ins Interface-Design aufnehmen.

3.6 Wirtschaftliche Bewertung

Der lokale Verarbeitungsstack nutzt ausschließlich Open-Source-Komponenten: MediaPipe, InsightFace, ONNX Runtime, Qdrant und FastAPI sind lizenzfrei (Lugaresi *u. a.*, 2019, S. 1–2). Damit fallen für die rechenlastigen Kernfunktionen keine Lizenz- oder GPU-Instanzkosten an. Wichtiger als der reine Kostenpunkt ist dabei, dass die biometrische Identifikation — die latenzsensibelste Operation — vollständig lokal läuft und dadurch keine Cloud-Abhängigkeit und keine laufenden API-Kosten pro erkanntem Gesicht entstehen.

SAP AI Core wird nur für drei Aufgaben genutzt: Gaze-Check und Begrüßung über Gemini 2.5 Flash sowie den Dialog über das Realtime-Sprachmodell. Der Gaze-Check läuft höchstens einmal pro vier Sekunden Kandidatensichtbarkeit, die Begrüßung nur bei einem vollständigen ACTIVE-Übergang — der Verbrauch bleibt damit auch im Dauerbetrieb gering. Die folgende Tabelle schätzt die API-Kosten für einen typischen 8-Stunden-Tag mit 30 Besuchern:

Komponente	Calls/Tag	Calls/Monat	Open-Source/ ONNX	AWS Rekognition	Azure Face API
Gaze-Check (Gemini 2.5 Flash)	45	1.350	identisch*	identisch*	identisch*
Begrüßung (Gemini 2.5 Flash)	30	900	identisch*	identisch*	identisch*
Dialog (gpt-4o Realtime)	90	2.700	identisch*	identisch*	identisch*
Gesichtserkennung (ArcFace/ONNX)	30	900	\$0,00	\$0,90	\$1,35

Tabelle 5: Geschätzte API-Kosten im 8-Stunden-Kiosk-Betrieb (30 Besucher/Tag). * Die Sprachmodell-Kosten fallen in allen Szenarien gleich an und werden daher nicht verglichen; verglichen wird nur die biometrische Identifikation. AWS Rekognition: \$0,001/Bild; Azure Face API: \$1,50/1.000 Transaktionen (laut öffentlichem Listenpreis).

In absoluten Zahlen ist die Ersparnis klein: Der Prototyp vermeidet Cloud-API-Kosten von rund \$0,90 (AWS Rekognition) bzw. \$1,35 (Azure Face API) pro Monat für die biometrische Identifikation. Der eigentliche Vorteil der lokalen Verarbeitung liegt damit nicht in der Ersparnis, sondern im Datenschutz (die Biometrie verlässt das interne Netz nicht, vgl. Kap. 3.5) und darin, dass für die kostenintensivste Komponente keine laufenden Gebühren anfallen und keine GPU-Hardware nötig ist. Die serverseitige Verarbeitung läuft im bestehenden SAP-Kubernetes-Cluster mit.

3.7 Nachhaltigkeitsaspekte

Ökologisch profitiert das System davon, dass die Inferenz über ONNX Runtime nur auf der CPU des Kiosk-Hosts läuft; ein dedizierter GPU-Server ist nicht nötig (vgl. Kap. 3.3). Damit entfällt der Energiebedarf für GPU-Instanzen, der bei Cloud-Alternativen den größten Verbrauchsposten ausmacht. Zusätzlich analysiert das System die Kameraframes nicht durchgehend, sondern im Takt von 1,0 s (FRAME_INTERVAL, vgl. Kap. 4.1), was die CPU-Dauerlast deutlich senkt.

Sozial betrifft die Nachhaltigkeit das Risiko demographisch ungleicher Fehlerraten: Gesichtserkennung kann je nach Trainingsdatensatz manche Bevölkerungsgruppen schlechter erkennen (Buolamwini und Gebru, 2018, S. 2–7) — ein Aspekt, den Kap. 7.1 einordnet. Der DSGVO-Rahmen (Art. 9) begrenzt den Einsatz ohnehin auf registrierte, einwilligende Personen (vgl. Kap. 3.5); für ein öffentliches Deployment wäre eine demographische Audit-Phase Teil des Produktivierungspfads.

4 Personenerkennung

Dieses Kapitel zeigt, wie die Personenerkennung im System umgesetzt ist: vom Kamerabild über die Gesichtsdetektion und die Prüfung, ob jemand wirklich mit dem Kiosk interagieren will, bis zu den Ereignissen, die das Frontend über Anwesenheit und Identität informieren. Es geht hier um das WIE der Umsetzung — warum welche Technologie gewählt wurde, steht in Kap. 3.

4.1 Gesichtsdetektion mit MediaPipe BlazeFace

Die Detektion nutzt drei konfigurierbare Filter (`MIN_DETECTION_CONFIDENCE` = 0,5, `MIN_FACE_WIDTH_RATIO` = 0,06 und `DETECTION_UPSCALE` = 2,5; vollständige Parameter siehe Anhang A2). Sie sortieren unerwünschte Detektionen aus. Der wichtigste ist `MIN_FACE_WIDTH_RATIO` mit 0,06. Er löst ein praktisches Problem: Eine Kamera sieht nicht nur die Person direkt vor dem Kiosk, sondern auch alle, die im Hintergrund vorbeigehen. Ohne diesen Filter würde jedes dieser Gesichter einen 4-Sekunden-Timer und damit eine Sitzung anstoßen — der Kiosk würde also ständig auf Passanten reagieren, die gar nicht stehenbleiben. Der Filter nutzt aus, dass ein weit entferntes Gesicht im Bild klein erscheint: Nimmt ein Gesicht weniger als 6 % der Bildbreite ein, steht die Person zu weit weg, um interagieren zu wollen, und wird aussortiert.

`DETECTION_UPSCALE` von 2,5 gleicht das Gegenteil aus: BlazeFace erkennt sehr kleine Gesichter schlechter (Bazarevsky u. a., 2019, S. 2–3). Der Frame wird deshalb vor der Detektion um Faktor 2,5 vergrößert, danach werden die gefundenen Boxen wieder auf die Originalgröße zurückgerechnet.

Die Detektion läuft nicht durchgehend, sondern im Takt von `FRAME_INTERVAL` = 1,0 s. Dieser Stichprobenansatz hält die CPU-Last niedrig und reicht zeitlich völlig aus (Lugaresi u. a., 2019, S. 1–2).

4.2 Gaze-Validierung via Vision-LLM

Der Gaze-Check läuft über Gemini 2.5 Flash (SAP AI Core) mit `temperature=0` und abgeschalteter interner Nachdenk-Phase (`thinking_budget=0`); Timeout ist `GAZE_TIMEOUT_SECS` = 9,0 s (vollständige Konfiguration siehe Anhang A2). Warum hier ein Vision-LLM statt einer klassischen Gaze-Estimation zum Einsatz kommt, ist in Kap. 3.2 und Kap. 2.1.2 begründet. Für die Umsetzung reicht ein binäres Urteil: Schaut die Person in die Kamera oder nicht?

Der verwendete Prompt lautet in der englischen Originalfassung: *„Look at this camera image from a kiosk. The camera is mounted at the top of the screen. Is the person’s face pointing toward the screen? Answer ‚yes‘ if the face is roughly frontal — head upright or only slightly tilted. Answer ‚no‘ if the head is clearly turned away, tilted far down, or tilted far up. Answer*

ONLY with ,yes' or ,no'. Bewusst wird nur nach Frontalität gefragt, nicht nach einer genauen Blickrichtung. Das Modell soll grob einschätzen, ob jemand interagieren will — und das robust genug, dass es bei verschiedenen Personen zuverlässig funktioniert.

Der Ablauf startet, sobald eine Person `CANDIDATE_SECS = 4,0 s` ununterbrochen erkannt wurde. Der Kameraframe wird mit OpenCV als JPEG (Qualitätsstufe 70) kodiert und mit dem Prompt an Gemini 2.5 Flash über SAP AI Core geschickt. Da der SDK-Aufruf synchron blockiert, läuft er in einem eigenen Hintergrund-Thread und hält den Detektions-Loop nicht auf.

Aus der Antwort ergeben sich drei Pfade. Bei „yes,“ wird der Übergang nach ACTIVE freigegeben (Details in Kap. 4.3), bei „no“ fällt die Maschine zurück nach IDLE. Überschreitet der Aufruf `GAZE_TIMEOUT_SECS = 9,0 s` oder schlägt er fehl, bleibt die Maschine im CANDIDATE-Zustand, statt nach IDLE zurückzufallen, und prüft im nächsten Frame erneut. So kostet ein kurzer Verbindungsfehler nicht gleich eine schon erkannte Interaktionsabsicht.

4.3 State Machine: IDLE → CANDIDATE → ACTIVE

Der Zustandsautomat kennt drei Zustände und wird über fünf Zeitparameter gesteuert (`CANDIDATE_SECS = 4,0 s`, `LEAVE_SECS = 10,0 s`, `GREETING_WAIT_SECS = 1,5 s` u. a.; vollständige Parameter siehe Anhang A2). Abbildung 2 zeigt die Übergänge.

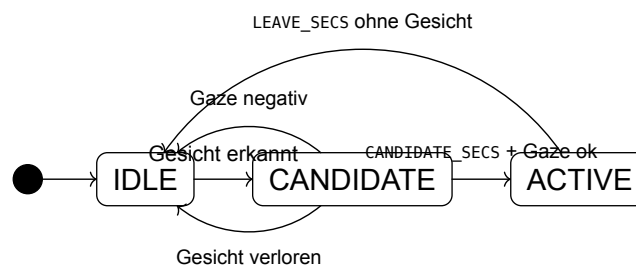


Abbildung 2: Zustandsdiagramm der PresenceStateMachine

Sobald `CANDIDATE_SECS = 4,0 s` überschritten sind, laufen zwei Dinge gleichzeitig an: die Gaze-Validierung und die biometrische Identifikation. Die Identifikation liefert nach rund 80 ms Name, Wiederkehr-Status, bevorzugte Sprache und den Gesprächskontext aus früheren Sitzungen. Mit diesen Daten startet parallel ein dritter Vorgang, der aus Kamerabild und Personendaten schon einen fertigen Begrüßungssatz erzeugt — vorsorglich, während der Gaze-Check noch läuft. Gebraucht wird er nur, wenn der Check positiv ausfällt.

Über den Übergang entscheidet dann der Gaze-Check. Fällt er positiv aus, geht der Zustand auf ACTIVE, die vorbereitete Begrüßung wird mit maximal `GREETING_WAIT_SECS = 1,5 s` Nachlauf eingesammelt und über das WebSocket-System gemeldet. Fällt er negativ aus, geht es zurück nach IDLE und die Begrüßung wird verworfen. Bei einem Timeout nach `GAZE_TIMEOUT_SECS = 9,0 s` bleibt der Zustand unverändert und der nächste Zyklus prüft erneut. Dieser Wieder-

helfall bekommt bewusst keinen eigenen Zustand: Ein Verbindungsfehler ist kein inhaltlicher Sonderfall, sondern nur eine technische Zwischenphase.

Der Rückweg von ACTIVE nach IDLE greift erst, wenn `LEAVE_SECS = 10,0 s` lang kein Gesicht der Person erkannt wurde. Kurze Unterbrechungen werden toleriert: Für jede Person merkt sich das System, seit wann sie nicht mehr gesehen wurde (`gone_since`). Taucht sie im nächsten Zyklus wieder auf, wird dieser Zeitstempel zurückgesetzt und die Sitzung läuft weiter. Erst nach vollen zehn Sekunden ohne ein einziges Wiedersehen endet die Sitzung. Normales Wegsehen oder kurzes Aus-dem-Bild-Treten beendet sie also nicht.

4.4 Paralleles Multi-Person-Tracking und Gruppen-Erkennung

Das Multi-Person-Tracking wird über `POSITION_MATCH_RADIUS = 120 px`, `SIMILARITY_THRESHOLD = 0,52` und das Sammel-Fenster `GROUP_ARRIVAL_WINDOW_SECS = 2,0 s` gesteuert (vollständige Parameter siehe Anhang A2).

Der `PersonTracker` führt für jede erkannte Person eine eigene Zustandsmaschine. Jede durchläuft den `IDLE → CANDIDATE → ACTIVE`-Zyklus aus Kap. 4.3 unabhängig von den anderen im Bild: `CANDIDATE`-Timer, Gaze-Ergebnis und Identität einer Person beeinflussen nie den Zustand einer anderen. Mehrere Personen können also gleichzeitig den `CANDIDATE`-Übergang durchlaufen.

Zur Zuordnung neuer Gesichter zu bekannten Personen prüft der Tracker zuerst eine schnelle, positionsbasierte Stufe; nur bei Uneindeutigkeit wird das teurere ArcFace-Embedding berechnet. Dieses zweistufige Tracking-by-Detection — Position zuerst, Erscheinungsbild nur bei Bedarf — passt gut zum Kiosk, wo Personen länger stehen und zwischendurch wegsehen (Bewley *u. a.*, 2016, S. 1–3), (Wojke, Bewley und Paulus, 2017, S. 1–3), (Barquero, Fernández und Hupont, 2020, S. 1–3). Schwellwerte und vollständiger Algorithmus stehen in Kap. 5.2.

Für Gruppen gilt das Sammel-Fenster `GROUP_ARRIVAL_WINDOW_SECS = 2,0 s`: Erreichen mehrere Personen innerhalb dieser Zeit die ACTIVE-Schwelle, fasst das System sie zu einem gemeinsamen `group_arrived`-Ereignis zusammen. IDLE-Einträge werden nach $\text{LEAVE_SECS} \cdot 6 \approx 60 \text{ s}$ entfernt, damit das Register nicht unbegrenzt wächst.

5 Biometrische Identifikation und Persistenz

Dieses Kapitel beschreibt, wie das System ein erkanntes Gesicht identifiziert und über mehrere Besuche hinweg speichert. Es setzt dort an, wo Kap. 4 aufhört: Ein aktives Gesicht liegt bereits vor. Der Aufbau folgt dem Verarbeitungsfluss — Kap. 5.1 berechnet das Gesichts-Embedding, Kap. 5.2 ordnet es über ein zweistufiges Tracking einer Person zu, und Kap. 5.3 speichert es dauerhaft und aktualisiert es bei jedem Besuch. Die Begründungen zur Modell- und Persistenzwahl stehen in Kap. 3.3 und Kap. 3.4.

5.1 Biometrische Identifikation mit InsightFace

Die Embedding-Berechnung nutzt das InsightFace-Modellpaket `buffalo_l` mit dem Modell `w600k_r50` (ArcFace ResNet50): 512-dimensionale Embeddings aus einem 112×112-px-Crop, berechnet über die ONNX Runtime (CPU) in 80 ms, bei einer LFW-Genauigkeit von 99,83 % und `SIMILARITY_THRESHOLD = 0,52` (vollständige Kennwerte siehe Anhang A2).

Sobald der Presence Service ein aktives Gesicht identifiziert hat, beginnt die Identifikation mit der Embedding-Berechnung. Ausgangspunkt ist die Bounding Box aus Kap. 4.1. Dieser Bildausschnitt wird in `face_id.py` auf 112×112 Pixel skaliert — die Eingabegröße, die das `w600k_r50`-Modell erwartet.

Aus diesem Crop berechnet das Modell ein L2-normiertes, 512-dimensionales Embedding — den biometrischen Fingerabdruck der Person im Embedding-Raum aus Kap. 2.2.1 (Schroff, Kalenichenko und Philbin, 2015, S. 1), (Taigman *u. a.*, 2014, S. 1). Als Ähnlichkeitsmaß dient der Kosinus-Score (Werte nahe 1,0 = hohe Übereinstimmung); der Schwellenwert `SIMILARITY_THRESHOLD` trennt „gleiche Person“, von „neue Person“.

Den Schwellenwert habe ich iterativ kalibriert. Startpunkt war der Literaturwert 0,65 für ArcFace-Verifikation (Deng *u. a.*, 2019, S. 4–5). Bei konstanter Beleuchtung liegen die Kosinus-Scores echter Wiedererkennungen typischerweise bei 0,72–0,85; ändert sich Licht oder Aussehen (z. B. eine neue Brille), fallen sie auf 0,53–0,58. Mit 0,65 werden solche legitimen Wiedererkennungen fälschlich als neue Person gewertet. Eine eigene Messreihe bestätigte das und führte zur Absenkung auf 0,52 — die konkreten Erkennungsraten stehen in Kap. 7.3.

Das Modell `w600k_r50` wurde mit ArcFace-Loss auf einem großen Gesichtsdatensatz trainiert (Details in Kap. 2.2) (Deng *u. a.*, 2019, S. 3). Die Inferenz läuft über die ONNX Runtime (Auswahl in Kap. 3.3) auf der CPU, ohne GPU-Server — 80 ms pro Embedding, schnell genug für den Erkennungszyklus mit `FRAME_INTERVAL = 1,0 s`.

5.2 Zweistufiges Tracking und Embedding-Stabilisierung

Der Tracker in `presence/tracker.py` ordnet in jedem Detektionszyklus die neu erkannten Gesichter den bereits bekannten Personen zu. Das geschieht zweistufig: Stage 1 ist ein günstiger Vorfilter, Stage 2 die teurere Embedding-Berechnung. Abbildung 3 zeigt den Ablauf:

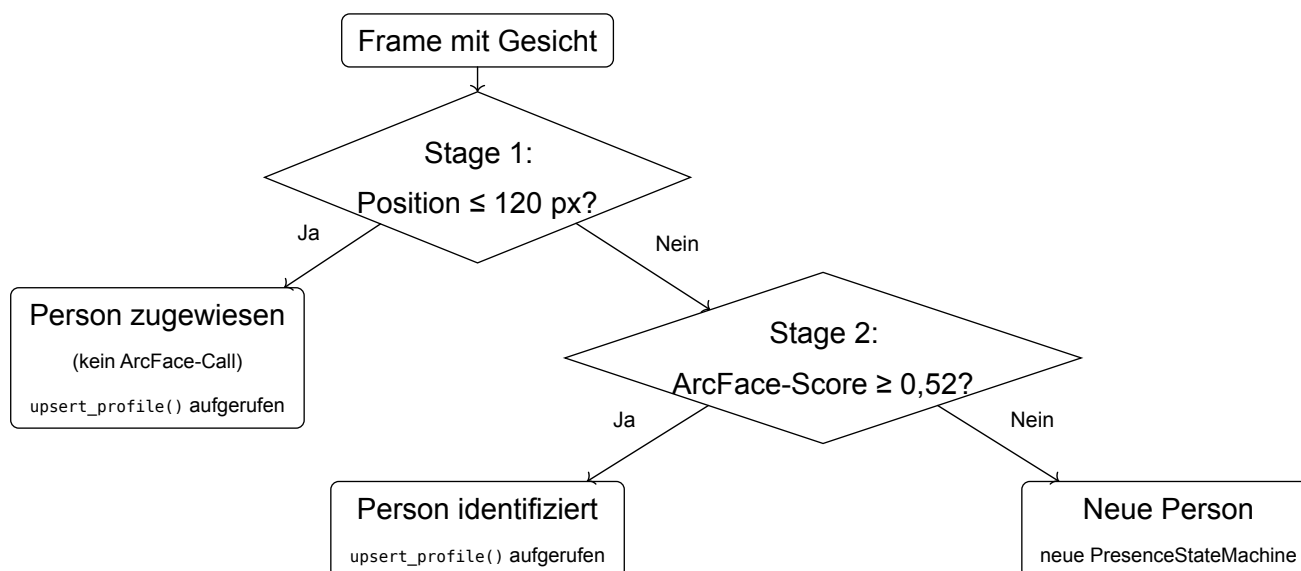


Abbildung 3: Zweistufiger Tracking-Algorithmus: Positions-Präfilter vor ArcFace-Matching

Stage 1 — Positions-Matching: Der Mittelpunkt der neuen Detektion wird mit den zuletzt bekannten Mittelpunkten aller aktiven Personen verglichen. Liegt der Abstand innerhalb von `POSITION_MATCH_RADIUS` (120 px), gilt die Detektion als Treffer und wird dieser Person zugewiesen — ganz ohne ArcFace-Embedding; bei mehreren Kandidaten gewinnt die nächstgelegene Person. Das setzt das Tracking-Prinzip aus Kap. 4.3 (SORT (Bewley *u. a.*, 2016, S. 1–3)) um. Die Koordinaten werden dabei auf den Original-Frame zurückgerechnet, da BlazeFace mit `DETECTION_UPSCALE = 2,5` arbeitet.

Stage 2 — ArcFace-Matching: Nur Detektionen ohne Stage-1-Treffer durchlaufen die Embedding-Berechnung. Der Kosinus-Score wird gegen alle gespeicherten Profile verglichen. Liegt der höchste Score über `SIMILARITY_THRESHOLD`, wird die Detektion der Person zugewiesen; liegt er darunter, entsteht eine neue `_TrackedPerson` mit eigener `PresenceStateMachine`. Diese Kombination aus Positions-Vorfilter und Embedding-Vergleich entspricht dem DeepSORT-Muster (Wojke, Bewley und Paulus, 2017, S. 1–3) und ordnet Personen auch dann zuverlässig zu, wenn sie kurz zur Seite blicken (Barquero, Fernández und Hupont, 2020, S. 3–4). Nach einem Treffer in Stage 1 oder Stage 2 wird `upsert_profile()` aufgerufen (Details in Kap. 5.3). Innerhalb einer Sitzung stabilisiert der Tracker das Embedding einer Person durch einen laufenden Mittelwert: Frame n trägt mit dem Gewicht $1/n$ bei (`_running_avg()`), sodass einzelne schlechte Frames das Sitzungs-Embedding nicht dominieren. Dieses stabilere Embedding ist die Grundlage für den sitzungsübergreifenden Upsert in Kap. 5.3.

5.3 Qdrant-Persistenz und EMA-Blending

Die Persistenzschicht basiert auf einem abstrakten FaceStore-Interface (Strategy-Pattern) mit zwei Backends, deren Auswahl per ENV-Variable `FACE_STORE_BACKEND` erfolgt: `SQLiteFaceStore` für die lokale Entwicklung und `QdrantFaceStore` für das Kubernetes-Deployment (Konfigurationsdetails siehe Anhang A2).

Das FaceStore-Interface trennt die Persistenz vom Identifikations-Algorithmus (Begründung vgl. Kap. 3.4): Über `FACE_STORE_BACKEND` wählt eine Factory zur Laufzeit zwischen `SQLiteFaceStore` und `QdrantFaceStore`. Qdrant sucht per HNSW-Index nach der ähnlichsten Person; liegt deren Kosinus-Score über `SIMILARITY_THRESHOLD`, gilt die Person als bekannt. Die `conversation_chunks`-Collection für die RAG-Embeddings wird in Kap. 6.2 beschrieben.

Bei jedem Besuch einer bekannten Person steht man vor einem Zielkonflikt. Ein festes, nie verändertes Template veraltet mit der Zeit: Ändert sich das Aussehen durch Licht, Frisur oder Alterung, sinken die Kosinus-Scores und die Person wird irgendwann fälschlich abgewiesen. Das Embedding einfach mit dem neuesten zu überschreiben, hat das umgekehrte Problem: Ein einziger schlechter Frame kann das Profil dauerhaft verschlechtern. Deshalb wird das gespeicherte Embedding nicht überschrieben, sondern bei jedem Besuch graduell aktualisiert:

$$e_{\text{neu}} = \text{normalize}((1 - \alpha) \cdot e_{\text{alt}} + \alpha \cdot e_{\text{aktuell}}), \quad \alpha = 0,2 \quad (3)$$

Der Faktor α steuert, wie stark der aktuelle Besuch das gespeicherte Profil verändert. Ich habe ihn empirisch bestimmt. Zuerst lag die Wahl bei $\alpha = 0,5$, also gleiches Gewicht für Alt und Neu. Im Testbetrieb bekamen dadurch einzelne schlechte Frames zu viel Einfluss und die Erkennungsscores sanken bei Folgebesuchen. Mit $\alpha = 0,2$ zählt das gespeicherte Langzeit-Embedding zu 80 % und der aktuelle Besuch nur zu 20 %. Das balanciert beide Anforderungen: stabil gegenüber einzelnen Ausreißern, aber offen für echte Veränderungen (neue Frisur, Brille) über mehrere Besuche — der Einfluss eines Besuchs sinkt nach fünf weiteren auf $(0{,}8)^5 \approx 33\% \sim$ % (eigene Beobachtung). Der Normierungsschritt stellt anschließend die L2-Norm wieder her (vgl. Kap. 5.1). Dieses Vorgehen entspricht dem etablierten Exponential-Weighted-Moving-Average-Prinzip adaptiver Erscheinungsmodelle (Dewan *u. a.*, 2016, S. 129–131), (Gardner, 2006, §2–3).

Zusammen mit dem Embedding speichert `upsert_profile()` bei jedem Besuch auch Metadaten (`visit_count`, `last_seen` und einen `metadata-Payload`). Diese ermöglichen es der Begrüßungslogik in Kap. 6.1, zwischen Erstbesuch und wiederkehrender Person zu unterscheiden.

6 Sitzungsübergreifende Personalisierung

Damit AIKO einen Besucher beim nächsten Mal wiedererkennt und an das letzte Gespräch anknüpfen kann, muss aus jeder Sitzung etwas Brauchbares übrig bleiben. Dieses Kapitel zeigt, wie das System Gespräche speichert und beim nächsten Besuch wieder abruft. Der Aufbau folgt dem Ablauf: Kap. 6.1 zeigt, wie ein Gespräch beim Verlassen einer Person zusammengefasst und in drei Kanäle abgelegt wird; Kap. 6.2 beschreibt, wie das System die passenden Fakten während der Sitzung wieder heraussucht und einspielt; Kap. 6.3 behandelt den Sonderfall Gruppen-Session. Warum überhaupt strukturierte Fakten statt der rohen Gesprächshistorie gespeichert werden, ist in Kap. 2.3 begründet.

6.1 Strukturierte Fakt-Extraktion

Verlässt eine erkannte Person das Blickfeld, löst das `person_left`-Event die Zusammenfassung der Sitzung aus. Das System legt die Informationen in drei getrennten Kanälen ab, die sich in Detailtiefe und Abrufweg unterscheiden:

Feld	Inhalt	Persistenz
summary	1–2 Sätze, die die Session kurz zusammenfassen (neutral, dritte Person)	1 Chunk mit [summary]-Tag; beim nächsten Besuch zu Sitzungsbeginn eingespielt (<code>_get_last_summary()</code>)
facts_sentences	Bis zu 5 kurze Einzelaussagen — aber nur, wo es sich lohnt: persönliche Vorlieben und Themen, nicht jeder Satz und keine Tool-Ergebnisse	Je 1 Chunk pro Satz; 384-dim Vektor für das RAG-Retrieval
facts	Stabile Kerndaten: name, location, language_preference, role, company	Als metadata-Payload im Profil; kein Chunk-Embedding

Tabelle 6: Dreikanaliges Gedächtnisdesign: Persistenzfelder der Fakt-Extraktion

Auslöser ist das `person_left`-Event, das die State Machine aus Kap. 4.3 an den Presence Listener meldet. Es ruft den Endpunkt `/api/summarize` auf, der die Gesprächshistorie an ein Sprachmodell übergibt. Ein fester JSON-Schema-Prompt zwingt das Modell, genau die drei Felder `summary`, `facts_sentences` und `facts` zurückzugeben (Brown *u. a.*, 2020, S. 7–9) — die Ausgabe ist damit direkt maschinenlesbar.

Die `facts_sentences` sind bewusst als kurze Einzelaussagen gespeichert — ein Fakt pro Chunk. Das ist für die spätere Suche wichtig: Ein einzelner, klarer Satz liegt im Vektorraum an

einer eindeutigen Stelle, während ein längerer Absatz mehrere Themen mischt und dadurch schwerer wiederzufinden ist (Xu, Szlam und Weston, 2022, S. 1). Dabei wird nicht jeder Satz zu einem Fakt: Das Modell extrahiert nur dort Aussagen, wo es einen persönlichen Bezug gibt — etwa eine Vorliebe oder ein wiederkehrendes Thema — und höchstens fünf davon. Das facts-Feld enthält dagegen stabile Kerndaten als Metadaten, die ohne Vektorsuche sofort für Begrüßung und Sitzungskonfiguration bereitstehen.

Das Aufteilen in drei Kanäle folgt dem Grundgedanken von MemGPT: ein kompakter Hauptkontext (hier `summary` beim Session-Start) neben einem externen Speicher (hier die `facts_sentences`-Chunks für die Suche während der Sitzung) (Packer *u. a.*, 2024, S. 1–3). Die komplette Gesprächshistorie mitzuschleppen, würde dagegen nur das Kontextfenster füllen, ohne dass sich einzelne Fakten gezielt abrufen ließen (vgl. (Liu *u. a.*, 2024, S. 4–6) und Kap. 2.3.1).

6.2 RAG-Retrieval und Kontext-Injektion

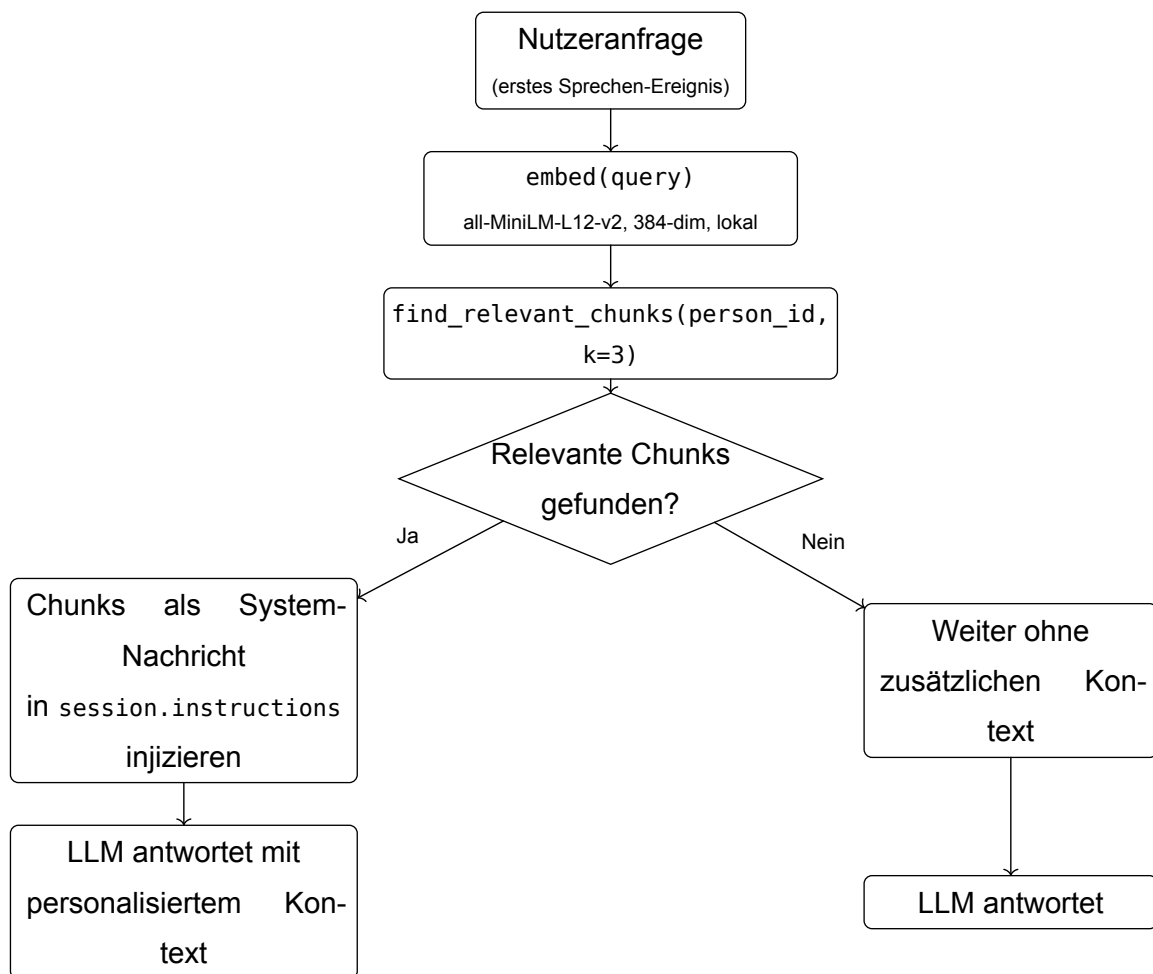


Abbildung 4: RAG-Retrieval-Flow: Embedding der Anfrage, Qdrant-Suche, Kontext-Injektion

Sobald der Nutzer in der laufenden Sitzung zum ersten Mal etwas sagt, startet der RAG-Abruf. Die Anfrage wird zunächst in einen Vektor umgewandelt: `embed()` in `backend/services/`

`embedder.py` lässt das Modell `all-MiniLM-L12-v2` lokal laufen und liefert einen 384-dimensionalen Vektor zurück. Das geht schnell und ohne Netzwerk (5 ms bei warmem Modell) — früher lief dieser Schritt als Remote-Aufruf über SAP AI Core und dauerte rund 900 ms. Das Modell baut auf der Sentence-BERT-Architektur auf und ist auf semantische Ähnlichkeitssuche in kompakten Vektorräumen ausgelegt (Reimers und Gurevych, 2019, S. 3984–3985); für uns zählte vor allem, dass es lokal läuft, kleine Vektoren liefert und treffsicher genug ist (Muennighoff *u. a.*, 2023, S. 2020–2022). Dieser Vektor ist der Suchschlüssel für `find_relevant_chunks(person_id, k=3)` in `presence/db/store.py`: Die Funktion durchsucht nur die Chunks dieser Person nach den drei ähnlichsten `facts_sentences`. Anfrage und gespeicherte Sätze liegen im selben 384-dimensionalen Raum, sodass sich thematische Nähe direkt messen lässt (Karpukhin *u. a.*, 2020, S. 1–2); die Suche läuft über den HNSW-Index von Qdrant (vgl. Kap. 3.4) (Malkov und Yashunin, 2020, S. 1–2).

Für die Aktualität sorgt zusätzlich die Injektion zum Sitzungsbeginn: Beim `person_arrived`-Event holt `_get_last_summary()` den neuesten `[summary]`-Chunk und schreibt ihn in `session.instructions`. So kennt das Sprachmodell schon ab dem ersten Wort grob den letzten Gesprächsstand des Besuchers — ohne dass dieser sein Vorwissen erneut erklären muss. Der Vorteil von RAG ist dabei, dass sich das Modell mit neuen, nutzerspezifischen Informationen anreichern lässt, ohne es neu zu trainieren (vgl. Kap. 2.3.3) (Lewis *u. a.*, 2020, S. 4).

Ein Beispiel aus dem SAP-Alltag macht den Nutzen greifbar: Ein SAP-Berater bereitet am Kiosk einen Kundentermin vor und sucht Infos für einen bestimmten Kunden — etwa die Müller AG, die eine S/4HANA-Migration plant. Das System merkt sich diesen Kontext (Kunde Müller AG, Thema Migration). Beim nächsten Besuch, kurz vor dem Folgetermin, fragt der Berater weitere Infos zu genau diesem Kunden. Das System erkennt ihn wieder, ruft den gespeicherten Kontext ab und spielt ihn in die laufende Sitzung ein. Der Assistent weiß dadurch direkt, für welchen Kunden die Antwort gedacht ist — der Berater muss seinen Hintergrund nicht noch einmal erklären.

Findet die Suche passende Chunks, spielt das Backend sie als Systemnachricht in die laufende Realtime-Session ein. Findet sie nichts, läuft die Sitzung ohne Zusatzkontext weiter — das Gespräch funktioniert trotzdem, nur eben ohne persönliche Anreicherung.

Der RAG-Abruf hat aber eine eingebaute Schwachstelle: Gibt die Suche thematisch unpassende Chunks zurück — weil die Anfrage zufällig ähnliche Vektorwerte wie ein fremdes Thema hat — landen diese als vermeintlich relevanter Kontext beim Sprachmodell, ohne dass das System den Fehler bemerkt. Solche stillen Fehltreffer sind ein bekanntes Problem bei RAG-Systemen (Karpukhin *u. a.*, 2020, S. 2). Zwei Designentscheidungen halten das Risiko klein: die kurzen Einzelaussagen der `facts_sentences` (ein Fakt pro Chunk, vgl. Kap. 6.1), sodass ein Chunk nie mehrere Themen mischt, und die Filterung auf `person_id`, sodass fremde Profile gar nicht in die Suche geraten. Ein Restrisiko bleibt — thematisch ähnliche, aber inhaltlich

unpassende Fakten derselben Person —, wird durch $k=3$ und die Ähnlichkeitsschwelle des Index aber begrenzt.

6.3 Gruppen-Sessions und History-Isolation

Sobald der Tracker aus Kap. 4.4 gleichzeitig mehrere Personen erkennt und sie innerhalb des Sammel-Fensters von zwei Sekunden zu einem `group_arrived`-Event zusammenfasst, wechselt das System in den Gruppen-Modus. In Einzel-Sessions wird beim `person_arrived`-Event der jüngste `[summary]`-Chunk via `_get_last_summary()` als `last_context` eingespielt, sodass das Gesprächsgedächtnis des letzten Besuchs von Anfang an bereitsteht. In Gruppen-Sessions passiert das bewusst nicht: keine individuelle Injektion, und das Fakt-Schema setzt das `facts`-Objekt per `_SUMMARIZE_PROMPT_GROUP` immer auf `{}` — für eine Gruppe werden also keine stabilen Kerndaten wie Name, Rolle oder Unternehmen gespeichert.

Der Grund dafür ist, dass sich in einer Gruppe nicht sauber zuordnen lässt, wer was gesagt hat. Reden Klaus und eine unbekannte Person gemeinsam — etwa über SAP-BTP-Migrationsoptionen —, kann das System hinterher nicht auseinanderhalten, welche Aussage von wem stammt. Würde es dieses Gruppengespräch trotzdem in Klaus' persönliches Profil schreiben, landeten fremde Aussagen und Vorlieben in seinen Fakt-Chunks — diese unbeabsichtigte Vermischung bezeichnet man als Cross-Contamination. Um das zu verhindern, speichert das System Gruppen-Chunks ohne Zuordnung zu einer Person und aktualisiert keine Einzelprofile. So bleiben biometrische Fakten datenschutzrechtlich nur eindeutig identifizierbaren Einzel-Sessions zugeordnet (Krivokuca Hahn und Marcel, 2023, S. 639–641) (vgl. Kap. 3.5).

Heikel sind vor allem die Übergänge zwischen Einzel- und Gruppenphase, denn dabei dürfen echte Einzel-Fakten nicht verloren gehen und keine Gruppenaussagen hineinrutschen. Spricht Klaus zunächst allein und kommt dann jemand dazu, sichert das System den Solo-Teil, bevor die Gruppenphase startet: `/api/summarize` wird mit dem Flag `was_group_session: false` aufgerufen, sodass Klaus' Einzel-Fakten aus der Solo-Phase in sein Profil einfließen — egal, was danach in der Gruppe gesprochen wird. Beim umgekehrten Übergang gilt dasselbe: Verlässt die zweite Person den Raum und Klaus redet allein weiter, merkt sich das System den Startpunkt dieser neuen Solo-Phase; beim abschließenden Summarize zählen nur Nachrichten ab diesem Punkt. So gelangen weder beim Start noch beim Ende einer Gruppenphase fremde Aussagen in Klaus' Einzelprofil.

Ob diese Trennung unter realen Bedingungen robust genug ist und wie oft das System überhaupt in den Gruppen-Modus wechselt, prüft Kap. 7.

7 Evaluation

Dieses Kapitel ordnet das System entlang von vier Dimensionen ein: Erkennungsgenauigkeit, Latenz, Robustheit und Personalisierungsqualität. Grundlage sind eigene Messungen und Beobachtungen aus dem Entwicklungs- und Testbetrieb. Kennzahlen aus früheren Kapiteln werden hier nur mit ihrem Ergebnis genannt.

Der Prototyp wurde über rund drei Monate im täglichen Bürobetrieb erprobt und dabei fortlaufend getestet. In dieser Zeit traten die praxisrelevanten Fälle wiederholt auf: Erst- und Wiedererkennung, Wechsel von Beleuchtung und Kamerastandort sowie Situationen mit mehreren Personen im Bild. Eine formale Probandenstudie mit standardisierter Versuchsanordnung war für diesen Prototyp nicht vorgesehen; die Aussagen beruhen auf dieser laufenden Beobachtung, nicht auf einer kontrollierten Erhebung. Zur Einordnung stellt die folgende Tabelle die in Kap. 3.1 definierten Anforderungen dem beobachteten Ergebnis gegenüber.

Anforderung	Ziel	Ergebnis	Status
A-01 Embedding-Latenz	≤ 100 ms	80 ms	erfüllt
A-02 LFW-Genauigkeit	≥ 99 %	99,83 %	erfüllt
A-03 Betrieb ohne GPU	CPU-only	CPU-only (ONNX Runtime)	erfüllt
A-04 keine Falschakzeptanz	im Betrieb	kein False-Accept über 3 Monate	erfüllt

Tabelle 7: Soll-Ist-Abgleich der Anforderungen A-01 bis A-04 aus Kap. 3.1

Alle vier Anforderungen wurden erfüllt. Die folgenden Abschnitte begründen diese Einordnung und benennen die jeweiligen Grenzen.

7.1 Erkennungsgenauigkeit

Das eingesetzte Modell liegt mit 99,83 % LFW-Genauigkeit auf dem Niveau etablierter Verfahren (Vergleich in Kap. 3.3). Für den Kiosk-Betrieb ist das ausreichend, denn ein Fehler bedeutet hier eine falsche Begrüßung und kein Sicherheitsrisiko. Über den gesamten dreimonatigen Testbetrieb trat kein einziger False-Accept auf — auch nicht bei Personen im Kamerabild, die nicht im System registriert waren: Keine von ihnen wurde fälschlich als bekannter Nutzer begrüßt.

Wie sich Schwellenwert und Erkennung im Feld verhalten, zeigt die eigene Score-Verteilung: Wiedererkennungen derselben Person erreichten unter konstantem Licht Kosinus-Werte von 0,72–0,85; nach einem Kamerastandort-Wechsel mit verändertem Licht sanken sie auf 0,53–0,58. Der Literatur-Startwert von 0,65 lag damit über diesen Grenzfällen, sodass nach dem Standortwechsel etwa die Hälfte der Wiedererkennungen fälschlich als neue Person gewertet wurde. Der auf 0,52 abgesenkte Betriebspunkt (vgl. Kap. 5.1) deckte auch diese Grenzfälle zuverlässig ab und hielt die Falschakzeptanz bei null. Für den Kiosk, wo eine Falschablehnung

nur störend, eine Falschakzeptanz aber schwerwiegender ist, ist dieser Betriebspunkt angemessen.

Zwei Grenzen sind zu nennen. Erstens bildet der LFW-Benchmark den Kiosk-Alltag nur teilweise ab: Die Bilder zeigen Prominente in frontalen Studioaufnahmen, nicht die wechselnde Bürobeleuchtung und Consumer-Kameras des realen Betriebs (Guo und Zhang, 2019, S. 4–5). Ein hoher LFW-Wert ist deshalb nur ein relativer Indikator; die eigentliche Feld-Validierung leistet die Schwellenwert-Kalibrierung am Einsatzstandort (vgl. Kap. 5.1), die bei einem fest platzierten Kiosk einmalig genügt. Zweitens fand der Testbetrieb in einem Büroumfeld statt und deckt damit keine breite demographische Vielfalt ab. Dass Gesichtserkennung je nach Trainingsdaten unterschiedliche Fehlerraten zwischen Bevölkerungsgruppen zeigen kann, ist bekannt (Buolamwini und Gebru, 2018, S. 2–7) (vgl. Kap. 3.7); für den internen SAP-Einsatz ist das vertretbar, ein öffentliches Deployment würde eine Prüfung über demographische Gruppen erfordern.

7.2 Systemlatenz

Die Zeit vom Erkennen einer Person bis zur personalisierten Begrüßung teilt sich in zwei Phasen: eine bewusst gesetzte Wartezeit und eine kurze, parallel laufende Rechenphase.

Phase	Inhalt	Dauer
Debounce (sequenziell) Entprellung — bewusste Wartezeit, die kurze Fehlauflösungen ausfiltert	CANDIDATE_SECS: Person muss durchgängig sichtbar sein, bevor die Verarbeitung startet	4,0 s
Verarbeitung (parallel)	Gaze-Check (2 s), Embedding (80 ms) und Begrüßungsgenerierung laufen gleichzeitig; der Gaze-Check dominiert	2 s
Summe	End-to-End bis personalisiertes Greeting	6 s

Tabelle 8: Systemlatenz: sequenzielle Wartezeit und parallele Verarbeitung

Die eigentliche Rechenlast ist nicht der Engpass: Embedding und Begrüßungsgenerierung laufen parallel zum Gaze-Check (Ablauf s. Kap. 4.3) und sind vor oder mit ihm fertig, sodass die Verarbeitungsphase vom 2 s dauernden Gaze-Check bestimmt wird. Der größte Anteil der End-to-End-Latenz ist damit die bewusst gesetzte Wartezeit von 4,0 s. Die resultierenden rund 6 s sind für den Kiosk akzeptabel: Wer aktiv interagieren möchte, steht ohnehin länger davor.

Der Gaze-Check zeigte im Testbetrieb folgendes Bild: Falsch als zugewandt erkannte Personen (False Positives) blieben klar in der Minderheit. Fälschlich abgewiesene aktive Nutzer (False Negatives) kamen etwas öfter vor, vor allem bei ungünstiger Kopfnäigung oder sehr

seitlichem Kamerawinkel. Da der Gaze-Check nur als Vorfilter wirkt, ist das tolerierbar: Der Nutzer muss sich lediglich nochmals direkt zum Kiosk wenden.

7.3 Robustheit

Faktor	Beobachtung	Gegenmaßnahme
Kopfdrehung > 30°	ArcFace-Score fällt von 0,80 auf 0,15	Positions-Vorfilter (Stage 1) vor dem ArcFace-Matching
Beleuchtungsvarianz	Bei verändertem Standort mit 0,65 rund die Hälfte der Ersterkennungen verpasst; nach Anpassung auf 0,52 zuverlässig	SIMILARITY_THRESHOLD-Kalibrierung (vgl. Kap. 5.1)
Multi-Person	Ca. 20 % der Sessions mit mehr als einer Person; der Gruppen-Mechanismus griff	GROUP_ARRIVAL_WINDOW + Duplicate-Merge

Tabelle 9: Robustheitsfaktoren und Gegenmaßnahmen im Testbetrieb

Die deutlichste Einschränkung ist die Winkelabhängigkeit: Bei Kopfdrehungen über 30° fällt der Ähnlichkeitswert von 0,80 auf 0,15 und damit unter jeden brauchbaren Schwellenwert (eigene Beobachtung). Aufgefangen wird das durch den Positions-Vorfilter des zweistufigen Trackings (Kap. 5.2), sodass der ArcFace-Score nur für tatsächlich zurückkehrende Personen ausschlaggebend ist — und dort ist er wieder zuverlässig. Die Beleuchtungsabhängigkeit fängt die Schwellenwert-Kalibrierung ab (Werte s. Tabelle 9). Gruppen-Sessions kamen in etwa einem Fünftel der Fälle vor; der GROUP_ARRIVAL_WINDOW-Mechanismus (vgl. Kap. 6.3) verhinderte Doppelbegrüßungen zuverlässig, eine Fehlzusammenführung von Gesprächshistorien wurde nicht beobachtet.

7.4 Personalisierungsqualität

Die drei vorigen Dimensionen bewerten technische Eigenschaften. Die in Kap. 1.3 formulierte Forschungsfrage zielt darüber hinaus auf die sitzungsübergreifende Personalisierung: ob das System für wiederkehrende Nutzer eine sinnvolle Gesprächskontinuität herstellt.

Diese Kontinuität funktionierte im Testbetrieb nachweislich. Das dreikanalige Gedächtnis (vgl. Kap. 6.1) injiziert beim Wiedererkennen den letzten Sitzungs-Summary und ruft beim ersten Sprechen thematisch passende Fakten aus dem personenspezifischen Index ab. Der Assistent konnte dadurch beim nächsten Besuch ohne erneute Erklärung auf das vorige Gesprächsthema Bezug nehmen; wechselte der Nutzer das Thema, blieb das Retrieval erwartungsgemäß ohne Treffer. Nicht Teil des Projektumfangs war eine breite Nutzerbefragung zur subjektiv wahrgenommenen Nützlichkeit; sie ist als Weiterarbeit denkbar (vgl. Kap. 8.2).

8 Fazit und Ausblick

8.1 Fazit

Die vorliegende Arbeit untersucht eine Forschungsfrage, die aus zwei Teilproblemen öffentlicher Kiosk-Systeme entsteht: einerseits dem bewussten Verzicht auf Login-Mechanismen (Porcheron *u. a.*, 2018, S. 3), andererseits dem fehlenden persistenten Gedächtnis großer Sprachmodelle über Sitzungsgrenzen hinweg (Zhang *u. a.*, 2018, S. 2388):

„Wie kann ein öffentlicher KI-Assistent durch kamerabasierte Gesichtserkennung mittels Deep-Learning-Embeddings wiederkehrende Nutzer identifizieren und eine sitzungsübergreifend personalisierte Konversationserfahrung bereitstellen?“

Die Frage lässt sich auf Ebene der technischen Machbarkeit positiv beantworten: Der Prototyp erkennt wiederkehrende Nutzer per kamerabasierter Gesichtserkennung zuverlässig wieder und stellt über strukturierte Fakt-Extraktion mit RAG-Retrieval eine personalisierte Konversation her, die über einzelne Sitzungen hinaus trägt. Wiedererkennung, Kontextabruf und sitzungsübergreifende Personalisierung sind technisch umgesetzt und im Testbetrieb bestätigt (vgl. Kap. 7).

Die Evaluation stützt diese Antwort in ihren Dimensionen unterschiedlich stark. Die Erkennungsgenauigkeit ist die belastbarste Aussage: Das Verfahren liegt auf dem Niveau etablierter Benchmarks, und für den Kiosk-Kontext — ein Fehler bedeutet eine falsche Begrüßung, kein Sicherheitsrisiko — ist die verbleibende Fehlerrate unkritisch. Die Latenz zeigt, dass nicht die biometrische Identifikation, sondern die bewusst gesetzten Wartezeiten der Interaktionslogik die Antwortzeit bestimmen; sie bleibt damit im akzeptablen Rahmen. Die Robustheit markiert die Grenze des Verfahrens — die Winkelabhängigkeit der Gesichtserkennung — die das zweistufige Tracking im laufenden Betrieb abfängt.

Der Beitrag der Arbeit liegt weniger in den einzelnen Verfahren — diese sind etabliert — als in ihrer Integration zu einem funktionierenden Gesamtsystem: Frontaldetektion, zweistufiges Tracking, Embedding-basierte Identifikation und ein Gesprächsgedächtnis greifen so ineinander, dass die in Kap. 1.1 beschriebene Personalisierungslücke öffentlicher Kiosk-Systeme geschlossen wird — ohne Login und ohne GPU-Server. Die Entkopplung über Microservices und das FaceStore-Interface hält die Lösung zugleich für andere Einsatzkontexte offen.

Drei Entscheidungen waren dabei besonders hilfreich: der Einsatz des Vision-LLM (Gemini 2.5 Flash) zur Interaktionsprüfung ohne nutzerspezifische Kalibrierung (vgl. Kap. 3.2); die parallele Vorberechnung der Begrüßung zeitgleich zum Gaze-Check, sodass die Generierungslatenz aus dem wahrnehmbaren Ablauf herausfällt (vgl. Kap. 4.3); und das dreikanalige Gedächtnis

aus `summary`, `facts_sentences` und `facts`, das kompakten Sitzungskontext, abrufbare Einzelfakten und stabile Kerndaten trennt und das System über Sitzungsgrenzen hinweg gesprächsfähig hält (vgl. Kap. 6.1).

Diese Antwort bezieht sich auf die technische Machbarkeit und stützt sich auf Benchmark-Werte und eigene Beobachtungen aus dem Testbetrieb, nicht auf eine kontrollierte Probandenstudie. Eine breitere empirische Erhebung unter realen Kiosk-Bedingungen ist als Weiterarbeit denkbar (vgl. Kap. 7.4).

8.2 Ausblick

Der Prototyp eröffnet mehrere Weiterentwicklungsrichtungen.

Die erste ist die Domänenübertragung: Das System ist nicht auf den SAP-Kiosk beschränkt, sondern auf alle Situationen übertragbar, in denen Nutzer bekannt sind, aber keine Login-Infrastruktur existiert (Porcheron *u. a.*, 2018, S. 4). Denkbare Felder sind die Pflege (Begrüßung beim Betreten eines Patientenzimmers ohne Geräteentsperrung), der Bildungsbereich (adaptive Lernsysteme, die die Lernhistorie ohne Login laden) und der Handel (Beratungssysteme, die Stammkunden wiedererkennen).

Die zweite Richtung ist die datenschutzrechtliche Produktivierung. Der Prototyp entstand unter internen Laborbedingungen; die in Kap. 3.5 genannten Voraussetzungen lassen sich in drei Schritte gliedern: (1) *Opt-in-Mechanismus* gemäß Art. 9 Abs. 2 lit. a DSGVO — ein Einwilligungs-Dialog, in dem Nutzer der Verarbeitung biometrischer Daten vor der ersten Registrierung aktiv zustimmen; (2) *Datenschutz-Folgenabschätzung* (DSFA) gemäß Art. 35 DSGVO (Europäisches Parlament und Rat der Europäischen Union, 2016, Art. 35) — eine dokumentierte Risikoanalyse der biometrischen Verarbeitung vor einem produktiven Rollout; (3) *Löschfunktion* gemäß Art. 17 DSGVO (Europäisches Parlament und Rat der Europäischen Union, 2016, Art. 17) — eine Funktion, über die gespeicherte Embeddings und Gesprächsprofile auf Anfrage vollständig gelöscht werden (Krivokuca Hahn und Marcel, 2023, S. 639–641). Diese Schritte erfordern keine Architekturumbauten, sondern ergänzende Implementierung auf Basis des bestehenden FaceStore-Interfaces (vgl. Kap. 3.4).

Die dritte Richtung ist die empirische Absicherung: Eine kontrollierte Studie mit größerer Probandengruppe würde die Genauigkeitsaussagen absichern und die Langzeitstabilität des EMA-Embedding-Gedächtnisses über Wochen und Monate validieren.

Anhang

A1: Übersicht eingesetzter KI-Werkzeuge

Werkzeug	Beschreibung der Nutzung
ChatGPT	• Verständnis von Grundbegriffen zu Gesichtserkennung und Embedding-Räumen (Kap. 2.2) • Erläuterung von Retrieval-Augmented Generation und Kontextfenster-Problematik (Kap. 2.3) • Identifikation und erste Sichtung von Literaturstellen zu ArcFace, RAG und Gaze-Estimation (Kap. 2) • Formulierungs- und Strukturierungshilfe für einzelne Textabschnitte (gesamt)
ChatPDF	• Zusammenfassung wissenschaftlicher Paper (u. a. ArcFace, FaceNet, MemGPT) zur schnelleren Einordnung (Kap. 2) • Auffinden relevanter Seitenzahlen in längeren Quell-PDFs für die Belege (Kap. 2, Kap. 5)
Microsoft CoPilot	• Korrektur- und Formulierungshilfe in Microsoft Word 365 (gesamt) • Übersetzung einzelner Textpassagen zwischen Deutsch und Englisch (gesamt)

Tabelle 10: Übersicht über die Nutzung von KI-basierten Werkzeugen

A2: Systemparameter

Parameter	Wert	Einheit	Kap.-Verweis
CANDIDATE_SECS	4,0	s	Kap. 4.3
LEAVE_SECS	10,0	s	Kap. 4.3
GAZE_TIMEOUT_SECS	9,0	s	Kap. 4.2
GREETING_WAIT_SECS	1,5	s	Kap. 4.3
FRAME_INTERVAL	1,0	s	Kap. 4.1
GROUP_ARRIVAL_WINDOW_SECS	2,0	s	Kap. 4.4
IDLE-Eviction (LEAVE_SECS·6)	≈60	s	Kap. 4.4
MIN_DETECTION_CONFIDENCE	0,5	—	Kap. 4.1
MIN_FACE_WIDTH_RATIO	0,06	—	Kap. 4.1
DETECTION_UPSCALE	2,5	Faktor	Kap. 4.1
POSITION_MATCH_RADIUS	120	px	Kap. 4.4/5.2
SIMILARITY_THRESHOLD	0,52	—	Kap. 5.1
EMA- α	0,2	—	Kap. 5.3

Tabelle 11: Konfigurierbare Parameter des Presence Service mit Standardwert, Einheit und Primärkapitel

Aspekt	Beschreibung
MIN_DETECTION_CONFIDENCE	MediaPipe-interne Schwelle für die Bounding-Box-Validierung — verwirft Detektionen unter diesem Konfidenzwert
MIN_FACE_WIDTH_RATIO	Mindestbreite eines erkannten Gesichts relativ zur Frame-Breite — filtert Hintergrundpersonen heraus (< 6 % des Frame-Bereichs)
DETECTION_UPSCALE	Frame wird vor Übergabe an MediaPipe um Faktor 2,5 vergrößert; Bounding-Boxen werden anschließend zurückgerechnet
Gaze-Modell	Gemini 2.5 Flash über SAP AI Core (thinking_budget=0, temperature=0, max_output_tokens=50)
Gaze-Eingabe	JPEG-kodiertes Kamerabild (Qualitätsstufe 70) plus Textaufforderung
Gaze-Ausgabe	Binäre Klassifikation: „yes„ → schaut in die Kamera, „no“ → nicht, Timeout/Fehler → None (Retry)

Tabelle 12: Wirkung der BlazeFace-Detektionsfilter und Gaze-Validator-Konfiguration

Aspekt	Eigenschaft
Modellpaket	InsightFace buffalo_l
Modell	w600k_r50 (ArcFace ResNet50)
Embedding-Dimension	512
Crop-Größe	112×112 px
Inference-Backend	ONNX Runtime (CPUExecutionProvider)
Latenz	80 ms pro Embedding-Berechnung
LFW-Genauigkeit	99,83 %
FaceStore-Typ	Abstrakte Basisklasse (Strategy-Pattern), Backend-Auswahl über ENV FACE_STORE_BACKEND
Backends	SQLiteFaceStore (lokal) / QdrantFaceStore (Kubernetes)
Vektordimensionen (Qdrant)	512-dim ArcFace-Embeddings / 384-dim RAG-Chunks
Ähnlichkeitsmaß	Kosinus-Distanz (HNSW-Index)

Tabelle 13: Kennwerte des InsightFace buffalo_l Erkennungsmodells und der FaceStore-Persistenzschicht

Literaturverzeichnis

Barquero, G., Fernández, C. und Hupont, I. (2020) „Long-Term Face Tracking for Crowded Video-Surveillance Scenarios“, in *International Joint Conference on Biometrics (IJCB)*. Verfügbar unter: <https://doi.org/10.1109/IJCB48548.2020.9304892>.

Bazarevsky, V. u. a. (2019) „BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs“. Verfügbar unter: <https://doi.org/10.48550/arXiv.1907.05047>.

Bewley, A. u. a. (2016) „Simple Online and Realtime Tracking“, in *2016 IEEE International Conference on Image Processing (ICIP)*, S. 3464–3468. Verfügbar unter: <https://doi.org/10.1109/ICIP.2016.7533003>.

Brown, T.B. u. a. (2020) „Language Models are Few-Shot Learners“, in *Advances in Neural Information Processing Systems (NeurIPS)*, S. 1877–1901. Verfügbar unter: <https://doi.org/10.48550/arXiv.2005.14165>.

Buolamwini, J. und Gebru, T. (2018) „Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification“, *Proceedings of Machine Learning Research*, 81, S. 1–15.

Cheng, Y. u. a. (2024) „Appearance-Based Gaze Estimation With Deep Learning: A Review and Benchmark“, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12), S. 7509–7528. Verfügbar unter: <https://doi.org/10.1109/TPAMI.2024.3393571>.

Deng, J. u. a. (2019) „ArcFace: Additive Angular Margin Loss for Deep Face Recognition“, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Verfügbar unter: <https://doi.org/10.48550/arXiv.1801.07698>.

Dewan, M.A.A. u. a. (2016) „Adaptive Appearance Model Tracking for Still-to-Video Face Recognition“, *Pattern Recognition*, 49, S. 129–151. Verfügbar unter: <https://doi.org/10.1016/j.patcog.2015.07.014>.

Dragoni, N. u. a. (2017) „Microservices: yesterday, today, and tomorrow“, in *Present and Ulterior Software Engineering*. Springer. Verfügbar unter: <https://doi.org/10.48550/arXiv.1606.04036>.

Europäisches Parlament und Rat der Europäischen Union (2016) *Verordnung (EU) 2016/679 des Europäischen Parlaments und des Rates vom 27.~April 2016 zum Schutz natürlicher Personen bei der Verarbeitung personenbezogener Daten, zum freien Datenverkehr und zur Aufhebung der Richtlinie 95/46/EG (Datenschutz-Grundverordnung)*. Verfügbar unter: <https://eur-lex.europa.eu/legal-content/DE/TXT/?uri=CELEX%3A32016R0679>.

Gardner, E.S., Jr. (2006) „Exponential smoothing: The state of the art—Part II“, *International Journal of Forecasting*, 22(4), S. 637–666. Verfügbar unter: <https://doi.org/10.1016/j.ijforecast.2006.03.005>.

- Guo, G. und Zhang, N. (2019) „A Survey on Deep Learning Based Face Recognition“, *Computer Vision and Image Understanding*, 189, S. 102805. Verfügbar unter: <https://doi.org/10.1016/j.cviu.2019.102805>.
- Guu, K. u. a. (2020) „REALM: Retrieval-Augmented Language Model Pre-Training“, in *Proceedings of the 37th International Conference on Machine Learning (ICML)*. PMLR (Proceedings of Machine Learning Research), S. 3929–3938. Verfügbar unter: <https://proceedings.mlr.press/v119/guu20a.html>.
- Jain, A.K., Ross, A.A. und Nandakumar, K. (2011) *Introduction to Biometrics*. New York: Springer. Verfügbar unter: <https://doi.org/10.1007/978-0-387-77326-1>.
- Johnson, J., Douze, M. und Jégou, H. (2019) „Billion-Scale Similarity Search with GPUs“, *IEEE Transactions on Big Data*, 7(3), S. 535–547. Verfügbar unter: <https://doi.org/10.48550/arXiv.1702.08734>.
- Karpukhin, V. u. a. (2020) „Dense Passage Retrieval for Open-Domain Question Answering“, in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, S. 6769–6781. Verfügbar unter: <https://doi.org/10.48550/arXiv.2004.04906>.
- Kellnhofer, P. u. a. (2019) „Gaze360: Physically Unconstrained Gaze Estimation in the Wild“, in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, S. 6911–6920. Verfügbar unter: <https://doi.org/10.1109/ICCV.2019.00701>.
- Krivokuca Hahn, V. und Marcel, S. (2023) „Biometric Template Protection for Neural-Network-Based Face Recognition Systems: A Survey of Methods and Evaluation Techniques“, *IEEE Transactions on Information Forensics and Security*, 18, S. 639–666. Verfügbar unter: <https://doi.org/10.1109/TIFS.2022.3228494>.
- Lewis, P. u. a. (2020) „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks“, in *Advances in Neural Information Processing Systems (NeurIPS)*, S. 9459–9474. Verfügbar unter: <https://doi.org/10.48550/arXiv.2005.11401>.
- Liu, N.F. u. a. (2024) „Lost in the Middle: How Language Models Use Long Contexts“, *Transactions of the Association for Computational Linguistics*, 12, S. 157–173. Verfügbar unter: <https://doi.org/10.48550/arXiv.2307.03172>.
- Liu, W. u. a. (2017) „SphereFace: Deep Hyperspherical Embedding for Face Recognition“, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Verfügbar unter: <https://doi.org/10.48550/arXiv.1704.08063>.
- Lugaresi, C. u. a. (2019) „MediaPipe: A Framework for Building Perception Pipelines“. Verfügbar unter: <https://doi.org/10.48550/arXiv.1906.08172>.
- Malkov, Y.A. und Yashunin, D.A. (2020) „Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs“, *IEEE Transactions on Pattern*

Analysis and Machine Intelligence, 42(4), S. 824–836. Verfügbar unter: <https://doi.org/10.1109/TPAMI.2018.2889473>.

Martin, R.C. (2017) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ: Pearson.

Muennighoff, N. u. a. (2023) „MTEB: Massive Text Embedding Benchmark“, in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. Dubrovnik, Croatia: Association for Computational Linguistics, S. 2014–2037. Verfügbar unter: <https://doi.org/10.18653/v1/2023.eacl-main.148>.

Packer, C. u. a. (2024) „MemGPT: Towards LLMs as Operating Systems“, in *Advances in Neural Information Processing Systems (NeurIPS)*, S. 1–30. Verfügbar unter: <https://doi.org/10.48550/arXiv.2310.08560>.

Porcheron, M. u. a. (2018) „Voice Interfaces in Everyday Life“, in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. ACM, S. 1–12. Verfügbar unter: <https://doi.org/10.1145/3173574.3174214>.

Radford, A. u. a. (2021) „Learning Transferable Visual Models From Natural Language Supervision“, in *Proceedings of the 38th International Conference on Machine Learning (ICML)*. PMLR (Proceedings of Machine Learning Research), S. 8748–8763. Verfügbar unter: <https://proceedings.mlr.press/v139/radford21a.html>.

Reimers, N. und Gurevych, I. (2019) „Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks“, in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, S. 3982–3992. Verfügbar unter: <https://doi.org/10.48550/arXiv.1908.10084>.

Rotter, P. (2008) „Making Biometrics Work: Technology Acceptance, Privacy Concerns, and Public Perceptions“, *Identity in the Information Society*, 1(1), S. 63–76. Verfügbar unter: <https://doi.org/10.1007/s12394-008-0006-7>.

Schroff, F., Kalenichenko, D. und Philbin, J. (2015) „FaceNet: A Unified Embedding for Face Recognition and Clustering“, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Verfügbar unter: <https://doi.org/10.48550/arXiv.1503.03832>.

Taigman, Y. u. a. (2014) „DeepFace: Closing the Gap to Human-Level Performance in Face Verification“, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, S. 1701–1708. Verfügbar unter: <https://doi.org/10.1109/CVPR.2014.220>.

Voigt, P. und Bussche, A. von dem (2017) „The EU General Data Protection Regulation (GDPR): A Practical Guide“, *Springer International Publishing* [Preprint]. Verfügbar unter: <https://doi.org/10.1007/978-3-319-57959-7>.

Wang, H. u. a. (2018) „CosFace: Large Margin Cosine Loss for Deep Face Recognition“, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Verfügbar unter: <https://doi.org/10.48550/arXiv.1801.09414>.

Wojke, N., Bewley, A. und Paulus, D. (2017) „Simple Online and Realtime Tracking with a Deep Association Metric“, in *2017 IEEE International Conference on Image Processing (ICIP)*, S. 3645–3649. Verfügbar unter: <https://doi.org/10.1109/ICIP.2017.8296962>.

Xu, J., Szlam, A. und Weston, J. (2022) „Beyond Goldfish Memory: Long-Term Open-Domain Conversation“, in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, S. 5650–5662. Verfügbar unter: <https://doi.org/10.48550/arXiv.2107.07567>.

Yin, P. u. a. (2024) „CLIP-Gaze: Towards General Gaze Estimation via Visual-Linguistic Model“, in *Proceedings of the AAAI Conference on Artificial Intelligence*, S. 6729–6737. Verfügbar unter: <https://doi.org/10.1609/aaai.v38i7.28496>.

Zhang, S. u. a. (2018) „Personalizing Dialogue Agents: I Have a Personality and it Matters“, in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, S. 2388–2402. Verfügbar unter: <https://doi.org/10.18653/v1/P18-1205>.

Zhang, X. u. a. (2015) „Appearance-Based Gaze Estimation in the Wild“, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, S. 4511–4520. Verfügbar unter: <https://doi.org/10.1109/CVPR.2015.7299081>.

Selbständigkeitserklärung

Ich versichere hiermit, dass ich die vorliegende Projektarbeit mit dem Thema

Biometrische Personalisierung eines KI-Assistenten

Entwicklung eines kamerabasierten Systems zur Gesichtserkennung und Nutzeridentifikation

selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt habe und diese bislang nicht veröffentlicht wurde. Des Weiteren versichere ich, dass die eingereichte elektronische Fassung mit der gedruckten Ausfertigung übereinstimmt.

Ort, Datum

Unterschrift

Erklärung und Nutzungsdokumentation zum Einsatz von KI-basierten Werkzeugen bei der Anfertigung von wissenschaftlichen Arbeiten als Prüfungsleistungen

Zur Verwendung KI-gestützter Werkzeuge erkläre ich in Kenntnis des Hinweisblatts „Hinweise zum Einsatz von KI-basierten Werkzeugen bei der Anfertigung von wissenschaftlichen Arbeiten als Prüfungsleistungen im prüfungsrechtlichen Kontext“ Folgendes:

- Ich habe mich aktiv über die Leistungsfähigkeit und Beschränkungen der in meiner Arbeit eingesetzten KI-Werkzeuge informiert.
- Bei der Anfertigung der Arbeit habe ich durchgehend eigenständig und beim Einsatz KI-gestützter Werkzeuge maßgeblich steuernd gearbeitet.
- Insbesondere habe ich die Inhalte entweder aus wissenschaftlicher oder anderen zugelassenen Quellen entnommen und diese gekennzeichnet oder diese unter Anwendung wissenschaftlicher Methoden selbst entwickelt.
- Mir ist bewusst, dass ich als Verfasser:in der Arbeit die volle Verantwortung für die in ihr gemachten Angaben und Aussagen trage.
- Soweit ich KI-gestützte Werkzeuge zur Erstellung der Arbeit eingesetzt habe, sind diese zu den Produktnamen, den formulierten Eingaben (Prompts), den Einsatzzwecken und der entsprechenden Seiten-/Bereichsreferenzierung auf die Arbeit im KI-Verzeichnis am Ende der Arbeit vollständig ausgewiesen und im Text belegt (z.B. als Fußnote).

Ort, Datum

Unterschrift